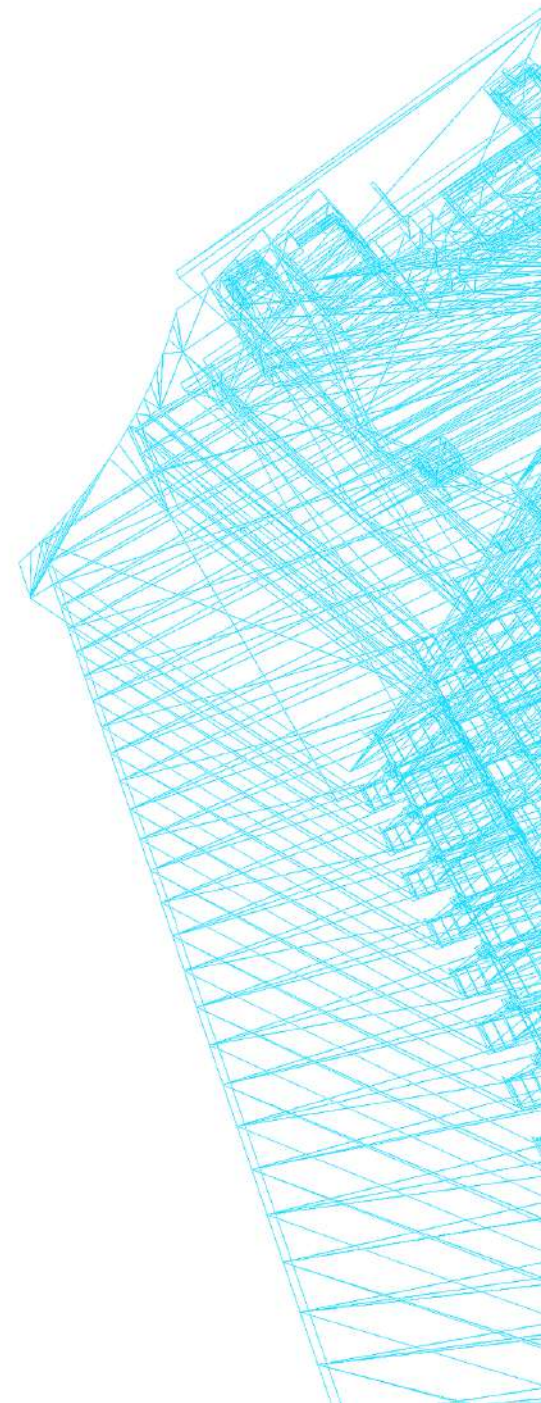


BACKTRACKING PARTE 2

João Pedro Oliveira Batisteli



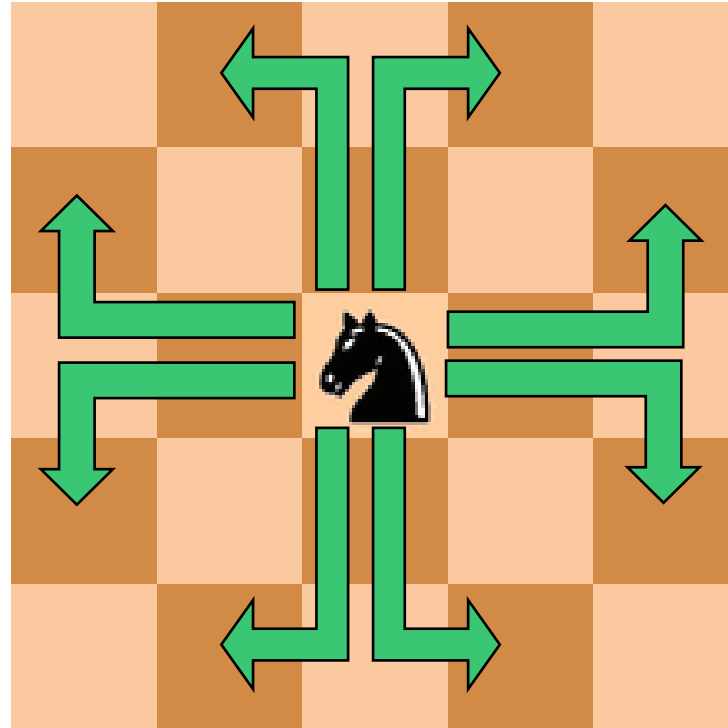
PASSEIO DO CAVALO



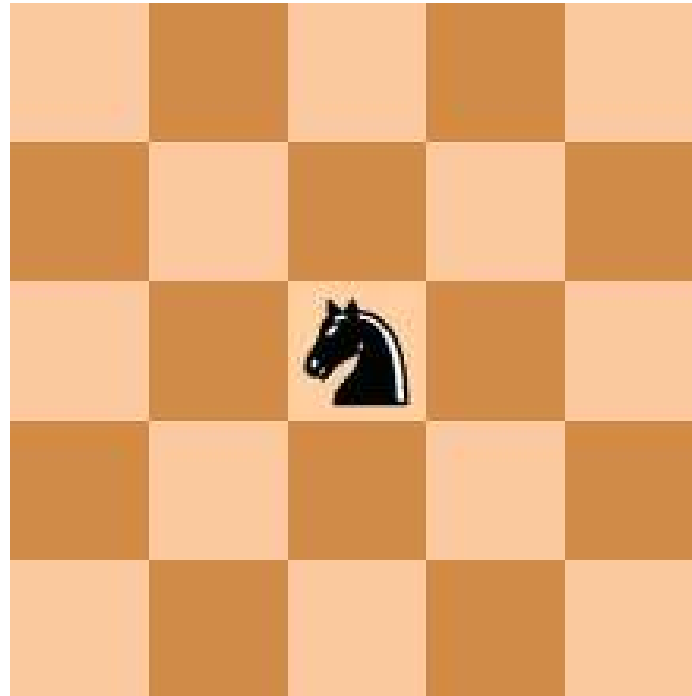
O PASSEIO DO CAVALO

- Dado um tabuleiro com $n \times n$ posições e um *cavalo* se movimentando segundo as regras do xadrez.
- **Problema:** A partir de uma posição inicial (x_0, y_0) , encontrar, se existir, um caminho que visite todas as casas do tabuleiro uma única vez e volte ao ponto inicial.

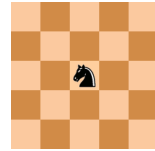
O PASSEIO DO CAVALO



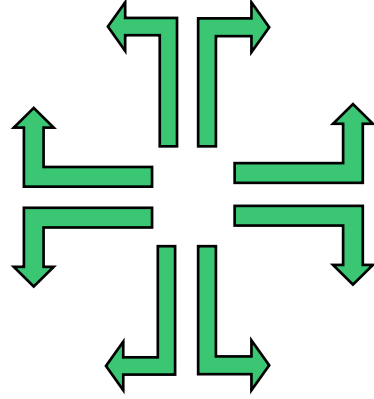
O PASSEIO DO CAVALO



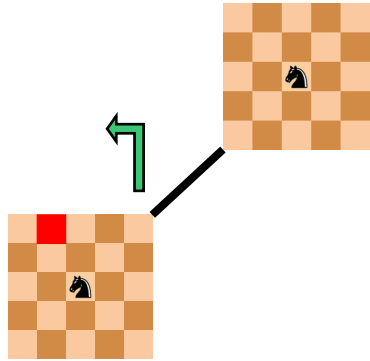
O PASSEIO DO CAVALO



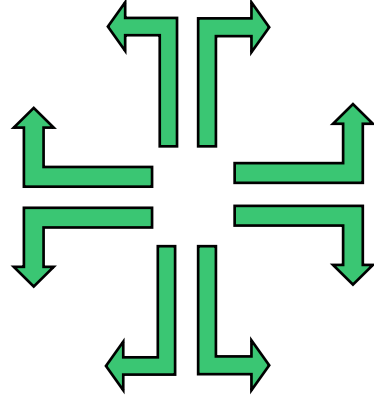
Conjunto de movimentos:



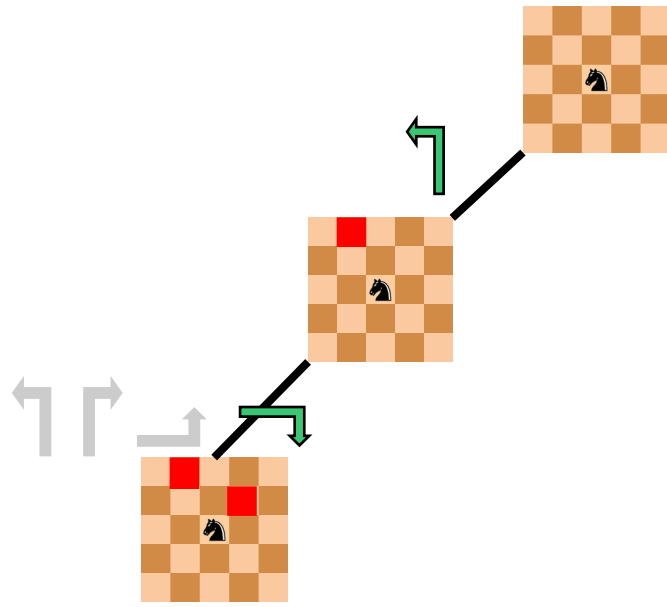
O PASSEIO DO CAVALO



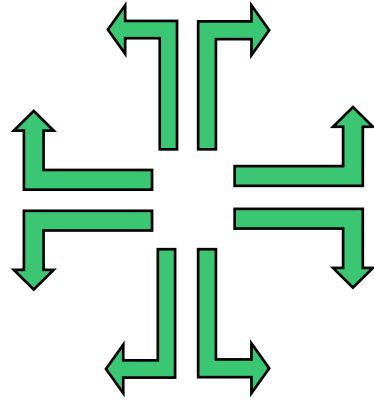
Conjunto de movimentos:



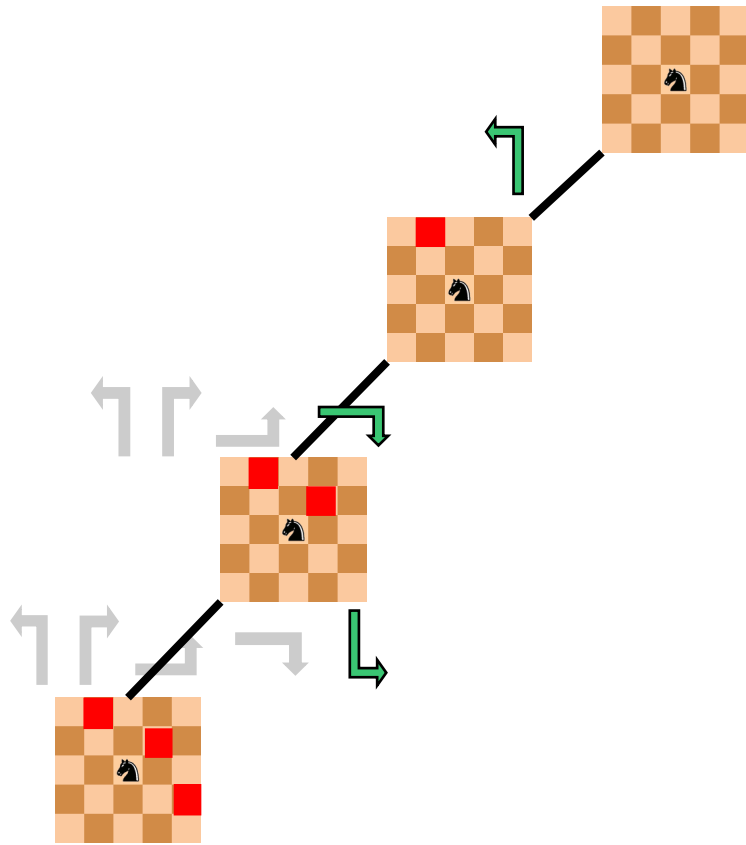
O PASSEIO DO CAVALO



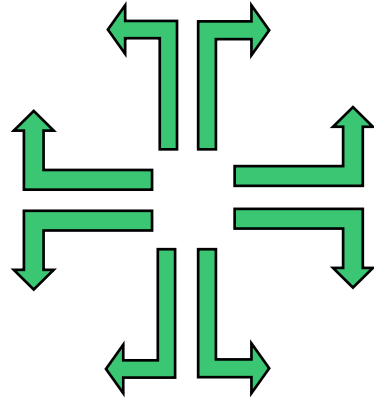
Conjunto de movimentos:



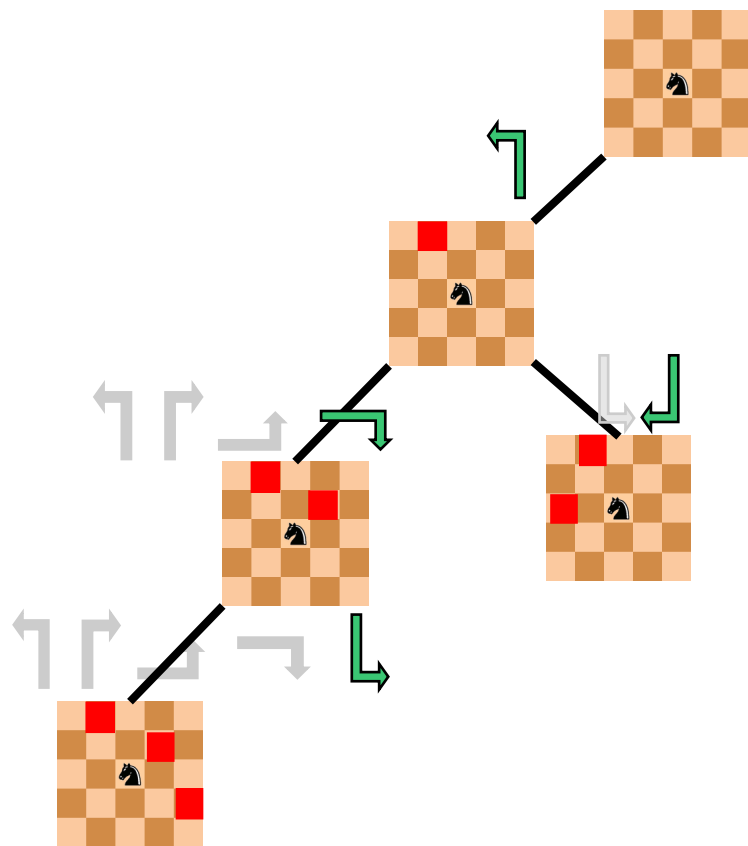
O PASSEIO DO CAVALO



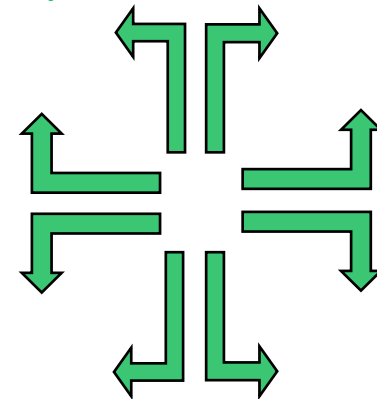
Conjunto de movimentos:

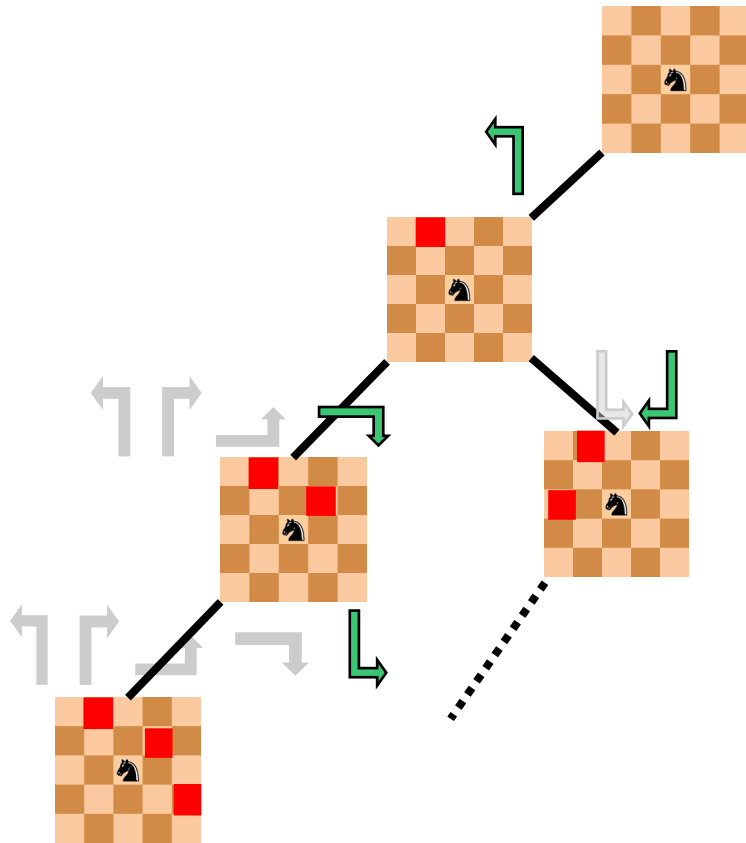


O PASSEIO DO CAVALO

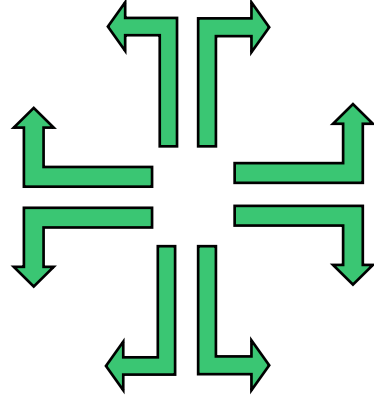


Conjunto de movimentos:



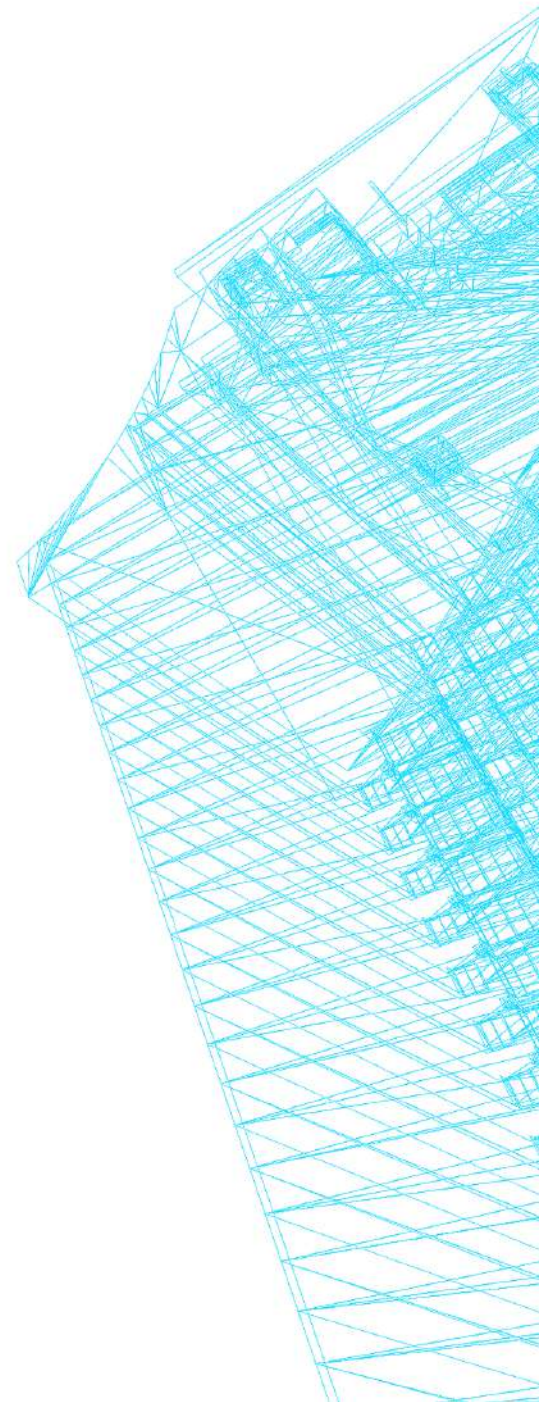


Conjunto de movimentos:



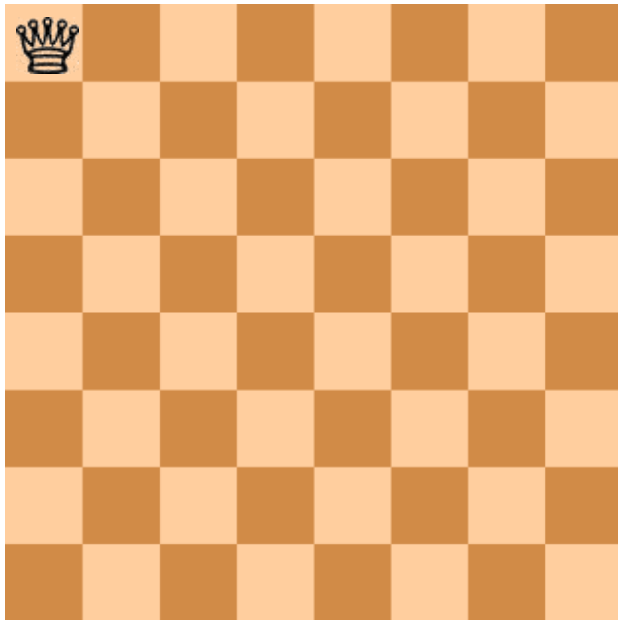
AS OITO RAINHAS

(ou mais ...)



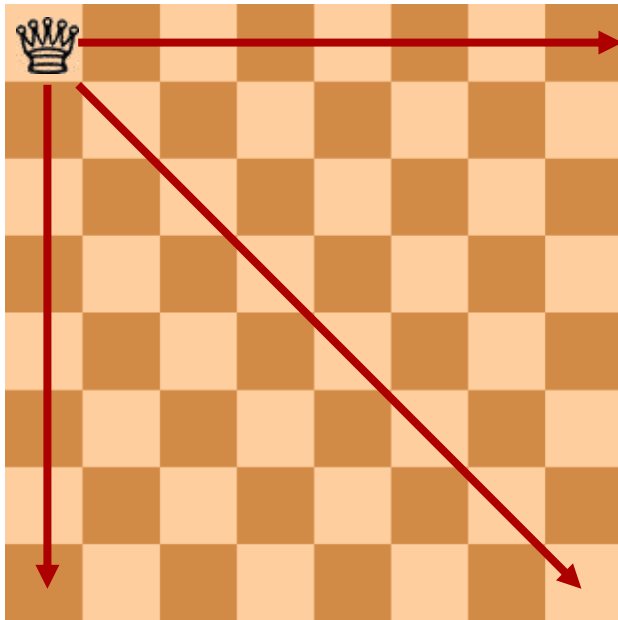
PROBLEMA DAS OITO RAINHAS

- **Rainha do xadrez:** pode se mover em qualquer direção, quantas casas desejar.



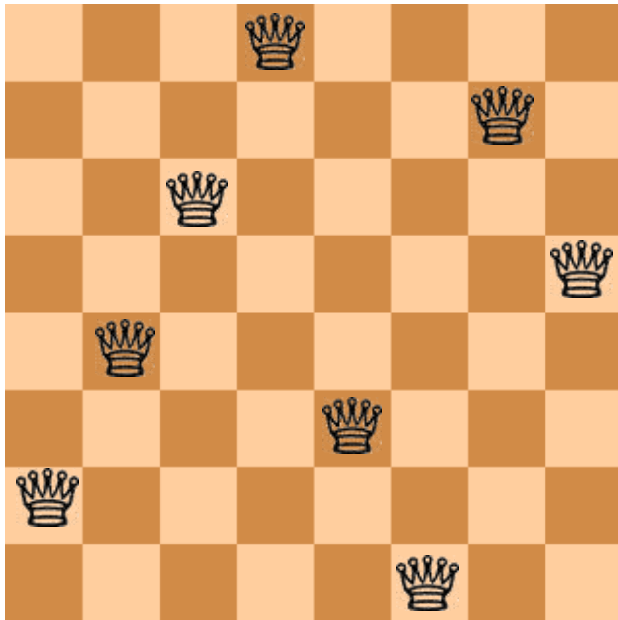
PROBLEMA DAS OITO RAINHAS

- **Rainha do xadrez:** pode se mover em qualquer direção, quantas casas desejar.



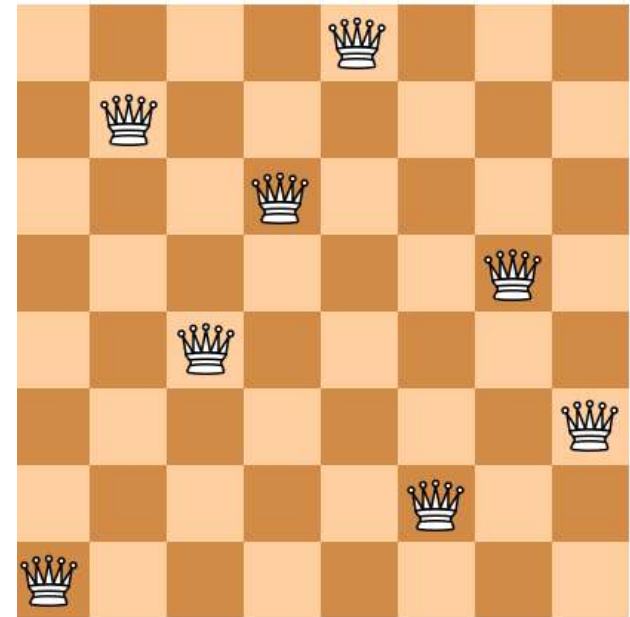
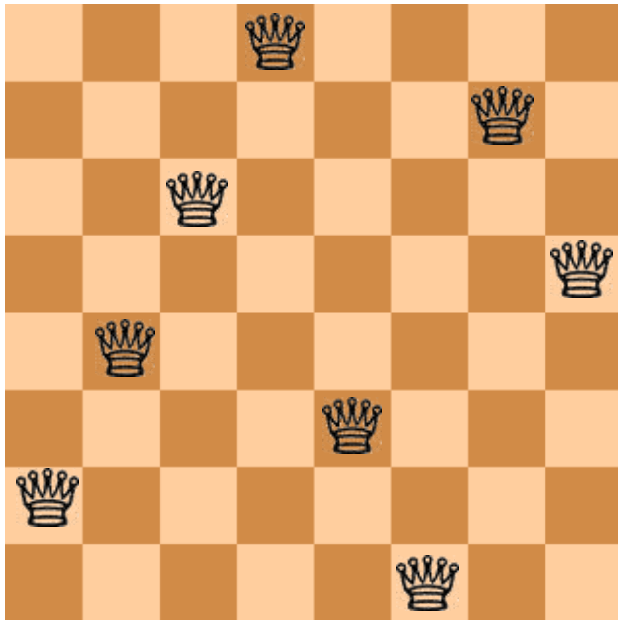
PROBLEMA DAS OITO RAINHAS

- **O problema:** colocar 8 rainhas em um tabuleiro de xadrez de modo que nenhuma possa atacar a outra.



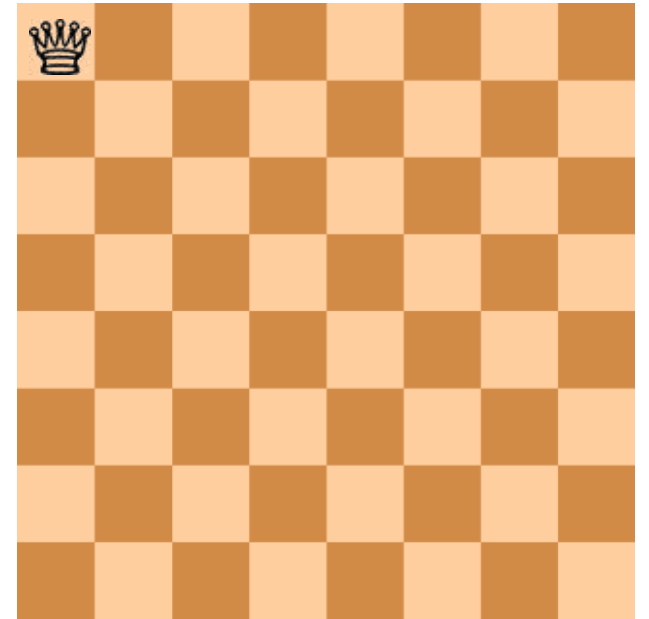
PROBLEMA DAS OITO RAINHAS

- **O problema:** colocar 8 rainhas em um tabuleiro de xadrez de modo que nenhuma possa atacar a outra.



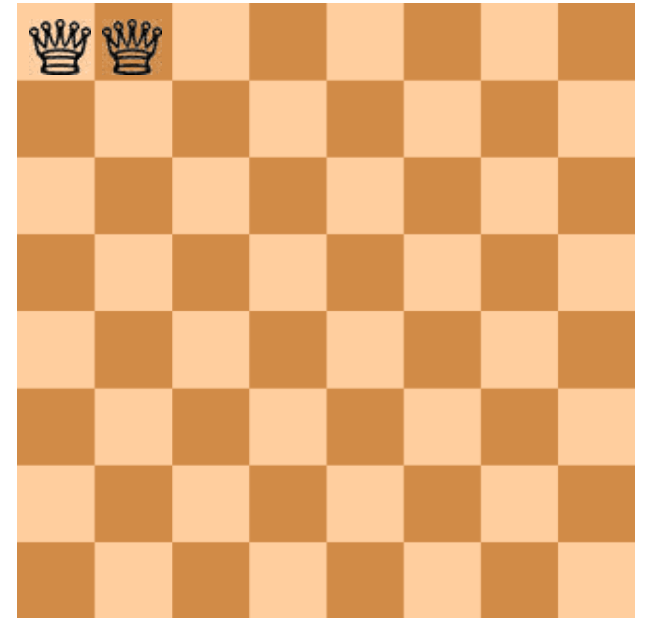
PROBLEMA DAS OITO RAINHAS

- *Backtracking:*
 1. Posiciona a primeira rainha em (0,0).



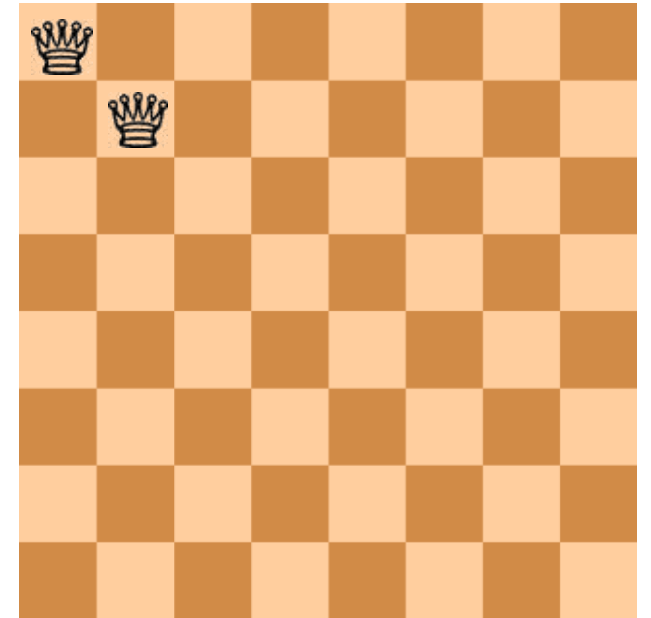
PROBLEMA DAS OITO RAINHAS

- *Backtracking:*
 1. Posiciona a primeira rainha em (0,0).
 2. Tentando (0, 1), verifica-se que está na mesma linha: solução inviável.



PROBLEMA DAS OITO RAINHAS

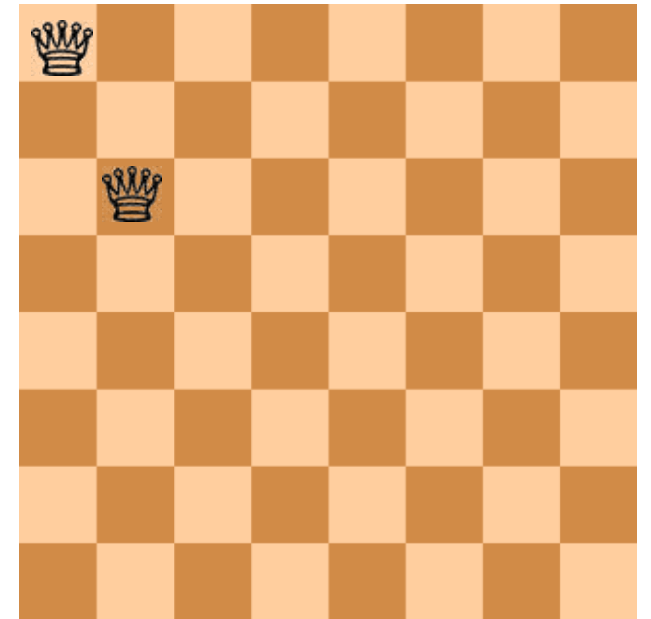
- *Backtracking:*
 1. Posiciona a primeira rainha em (0,0).
 2. Tentando (0, 1), verifica-se que está na mesma linha: solução inviável.
 3. Tentando (1, 1), verifica-se que está na mesma diagonal: solução inviável.



PROBLEMA DAS OITO RAINHAS

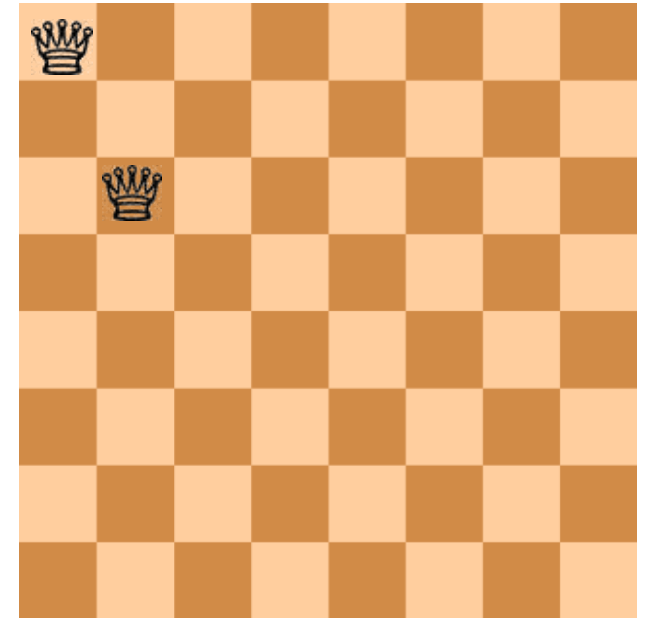
- *Backtracking:*

1. Posiciona a primeira rainha em (0,0).
2. Tentando (0, 1), verifica-se que está na mesma linha: solução inviável.
3. Tentando (1, 1), verifica-se que está na mesma diagonal: solução inviável.
4. Escolhe a próxima opção (2, 1) e avança para a terceira rainha.



PROBLEMA DAS OITO RAINHAS

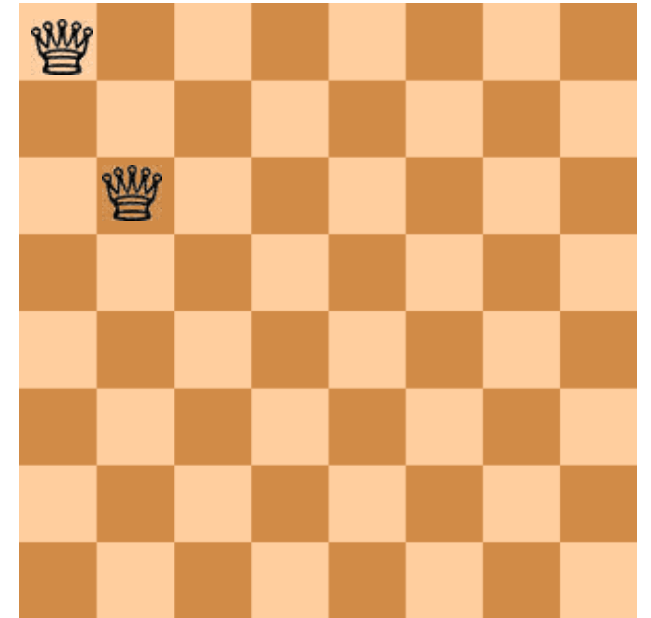
- *Backtracking:*
 1. Posiciona a primeira rainha em (0,0).
 2. Tentando (0, 1), verifica-se que está na mesma linha: solução inviável.
 3. Tentando (1, 1), verifica-se que está na mesma diagonal: solução inviável.
 4. Escolhe a próxima opção (2, 1) e avança para a terceira rainha.
 5. Eventualmente, o retrocesso pode chegar a raiz (solução inicial).



PROBLEMA DAS OITO RAINHAS

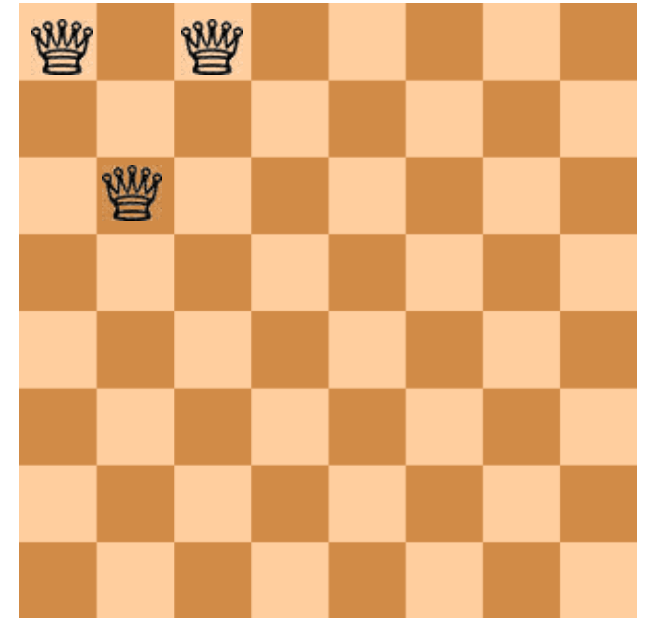
- *Backtracking:*

1. **Posiciona a primeira rainha em (0,0).**
2. Tentando (0, 1), verifica-se que está na mesma linha: solução inviável.
3. Tentando (1, 1), verifica-se que está na mesma diagonal: solução inviável.
4. Escolhe a próxima opção (2, 1) e avança para a terceira rainha.
5. Eventualmente, o retrocesso pode chegar a **raiz** (solução inicial).



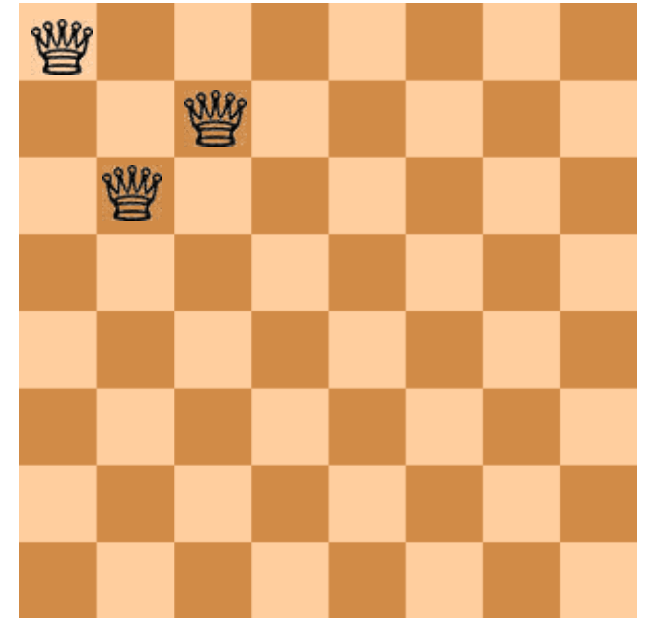
PROBLEMA DAS OITO RAINHAS

- *Backtracking:*
 1. Tentando (0, 2), verifica-se que está na mesma linha: solução inviável.



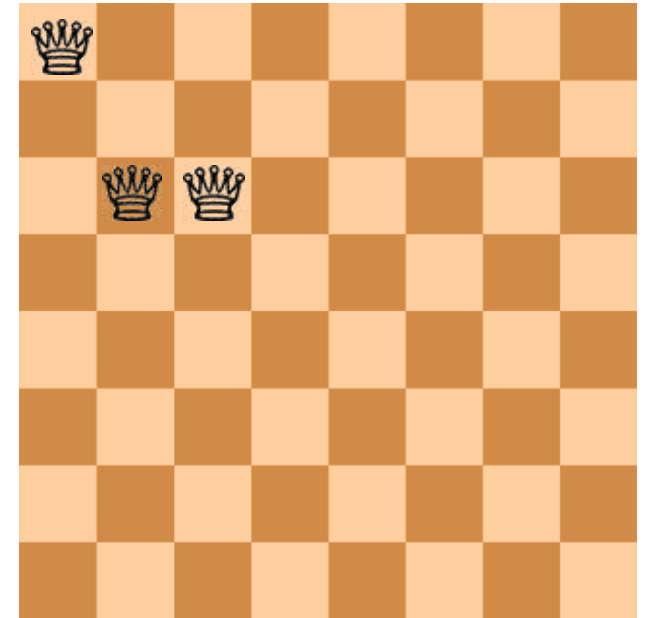
PROBLEMA DAS OITO RAINHAS

- *Backtracking:*
 1. Tentando (0, 2), verifica-se que está na mesma linha: solução inviável.
 2. Tentando (1, 2), verifica-se que está na mesma diagonal: solução inviável.



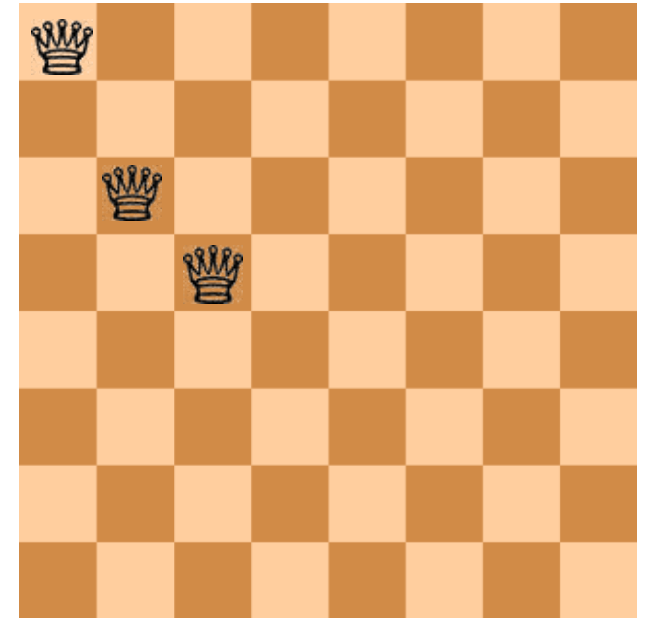
PROBLEMA DAS OITO RAINHAS

- *Backtracking:*
 1. Tentando (0, 2), verifica-se que está na mesma linha: solução inviável.
 2. Tentando (1, 2), verifica-se que está na mesma diagonal: solução inviável.
 3. Tentando (1, 3), verifica-se que está na mesma linha: solução inviável.



PROBLEMA DAS OITO RAINHAS

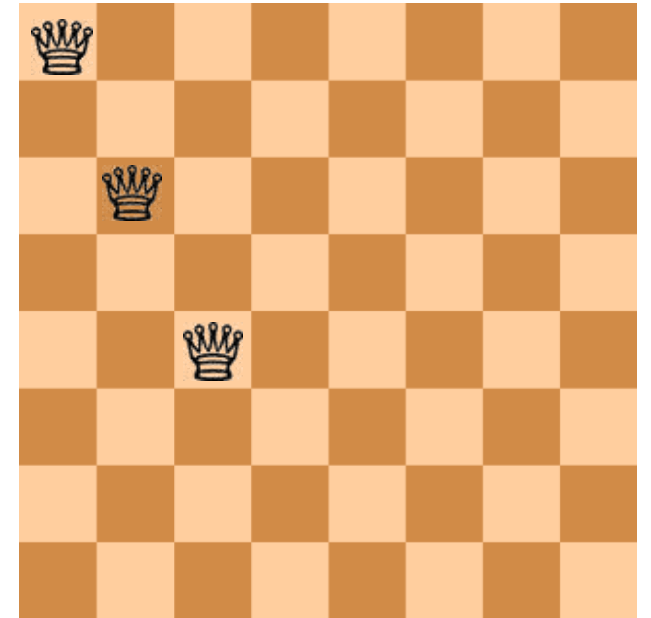
- *Backtracking:*
 1. Tentando (0, 2), verifica-se que está na mesma linha: solução inviável.
 2. Tentando (1, 2), verifica-se que está na mesma diagonal: solução inviável.
 3. Tentando (1, 3), verifica-se que está na mesma linha: solução inviável.
 4. Tentando (1, 4), verifica-se que está na mesma diagonal: solução inviável.



PROBLEMA DAS OITO RAINHAS

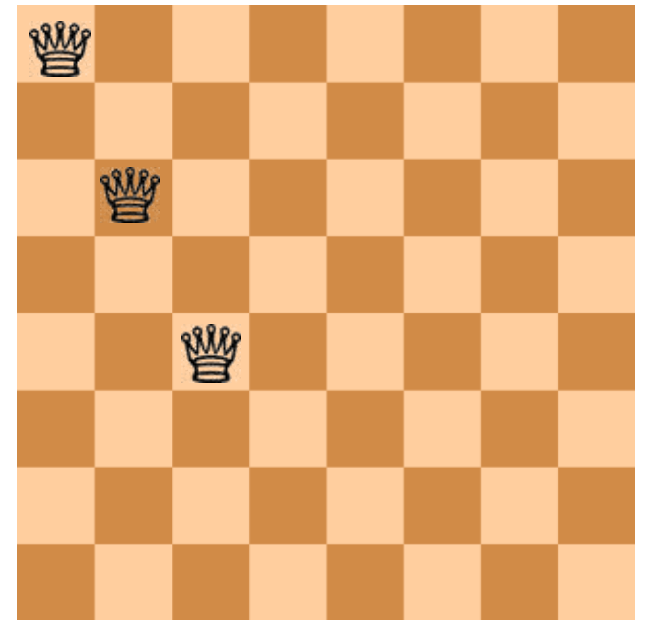
- *Backtracking:*

1. Tentando (0, 2), verifica-se que está na mesma linha: solução inviável.
2. Tentando (1, 2), verifica-se que está na mesma diagonal: solução inviável.
3. Tentando (1, 3), verifica-se que está na mesma linha: solução inviável.
4. Tentando (1, 4), verifica-se que está na mesma diagonal: solução inviável.
5. Tentando e posicionando em (1,5).



PROBLEMA DAS OITO RAINHAS

É uma boa abordagem?

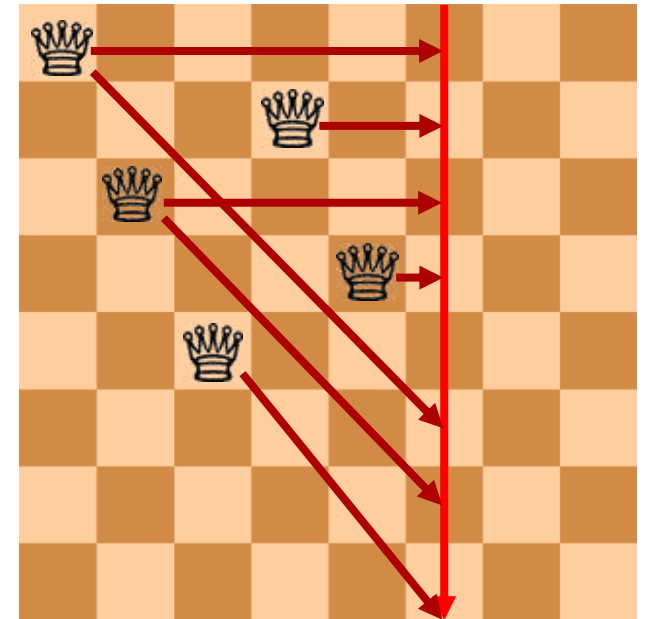


PROBLEMA DAS OITO RAINHAS

- *Backtracking:*

1. Eventualmente, pode acontecer de não restar solução viável.

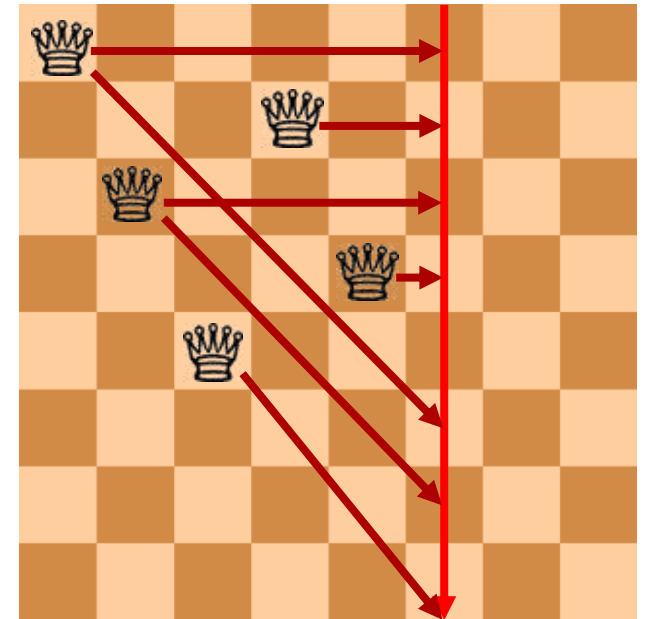
Posicionar a rainha na coluna 5



PROBLEMA DAS OITO RAINHAS

- *Backtracking*:
 1. Eventualmente, pode acontecer de não restar solução viável.
 2. Retrocesso para último passo e nova tentativa.

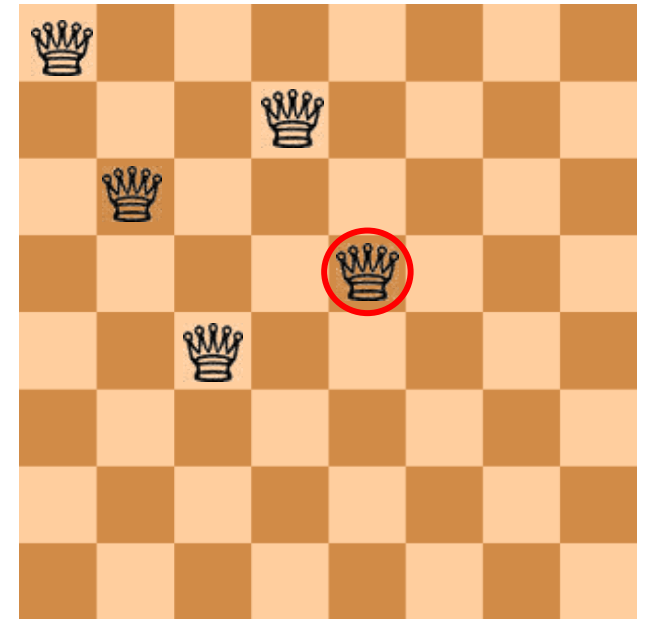
Posicionar a rainha na coluna 5



PROBLEMA DAS OITO RAINHAS

- *Backtracking*:
 1. Eventualmente, pode acontecer de não restar solução viável.
 2. Retrocesso para último passo e nova tentativa.

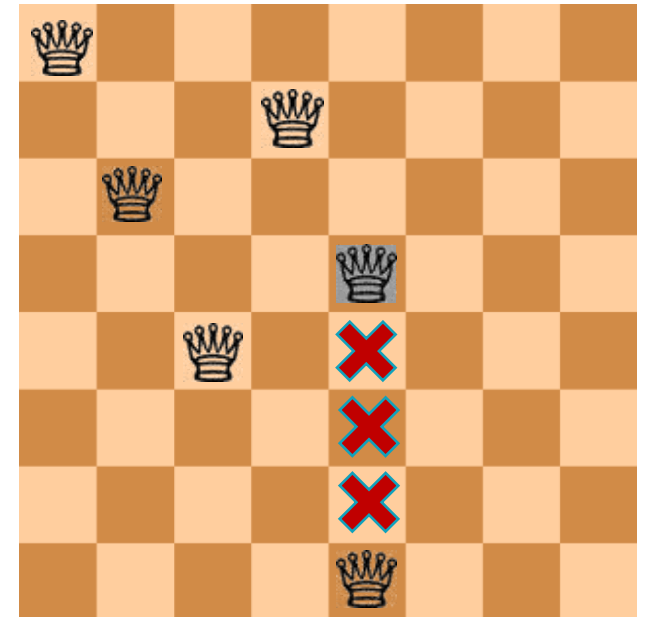
Posicionar a rainha na coluna 5



PROBLEMA DAS OITO RAINHAS

- *Backtracking*:
 1. Eventualmente, pode acontecer de não restar solução viável.
 2. Retrocesso para último passo e nova tentativa.
 3. Em último caso, pode ser necessário retroceder até a raiz =(

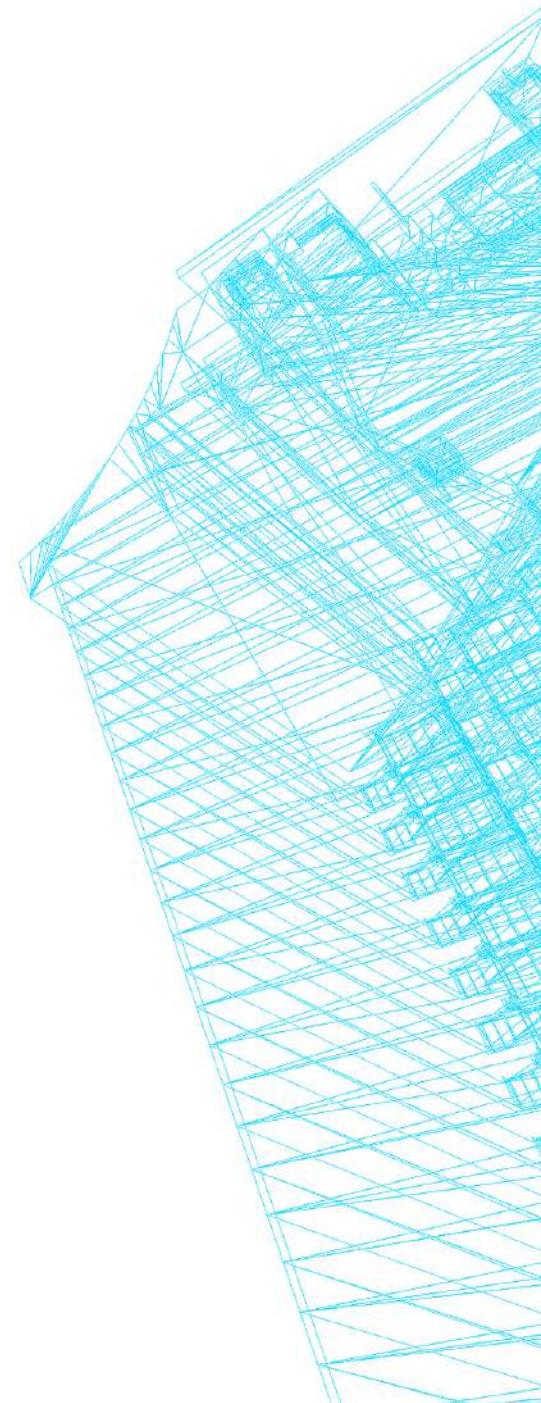
Posicionar a rainha na coluna 5



PROBLEMA DAS N RAINHAS

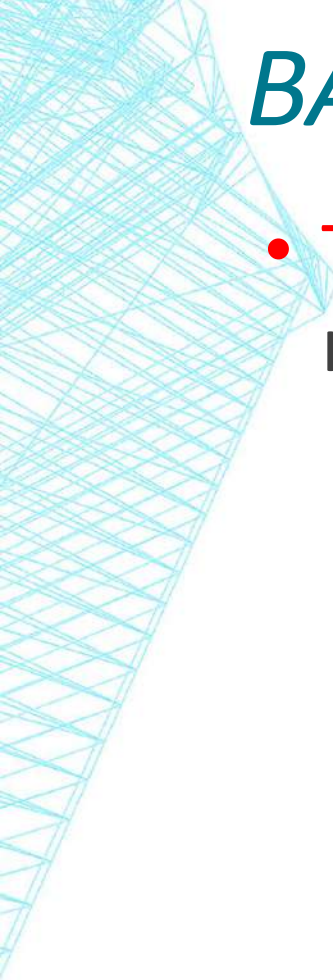
- Colocar N rainhas em um tabuleiro de $N \times N$ de modo que nenhuma possa atacar outra.
 - Para algumas instâncias de n , o problema não tem solução.

CRITÉRIOS DE PODA



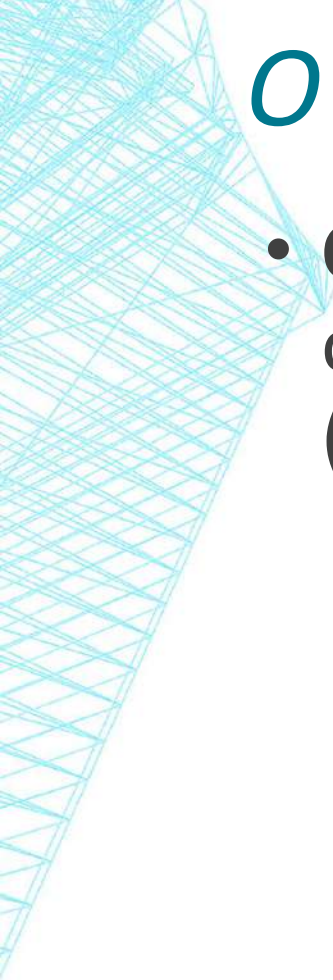
BACKTRACKING E PODA

- **Cenário Ideal (Alta Eficiência):**
 - Ocorrem muitas falhas prematuras (condições inaceitáveis).
 - A poda elimina grandes "galhos" da árvore de busca.
 - O algoritmo resolve problemas que seriam intratáveis por força bruta pura.
- **Pior Caso (Gargalo Computacional):**
 - As regras do problema geram muitas soluções candidatas ou "falsos positivos" que só falham no final.
 - A árvore de busca é percorrida quase em sua totalidade.
 - O comportamento e o tempo de execução se aproximam da Força Bruta, assumindo complexidades exponenciais ($O(2^n)$) ou fatoriais ($O(n!)$).



BACKTRACKING E PODA

- **Tentativa de fuga do pior caso de tempo:** estabelecer critérios mais agressivos de poda.



O MECANISMO DE PODA (PRUNING)

- **Conceito:** Utilização de uma função de avaliação para reduzir o espaço de busca, cortando antecipadamente subproblemas (ramos da árvore) que não levarão ao objetivo.

O MECANISMO DE PODA (PRUNING)

Conceito: Utilização de uma função de avaliação para reduzir o espaço de busca, cortando antecipadamente subproblemas (ramos da árvore) que não levarão ao objetivo.

Tipos de Critérios:

1) Poda Garantida (Exata):

- Baseia-se em restrições matemáticas e limites lógicos do problema.
- **Garantia:** É 100% segura. Jamais descarta um caminho que contenha a solução ótima.

O MECANISMO DE PODA (PRUNING)

Conceito: Utilização de uma função de avaliação para reduzir o espaço de busca, cortando antecipadamente subproblemas (ramos da árvore) que não levarão ao objetivo.

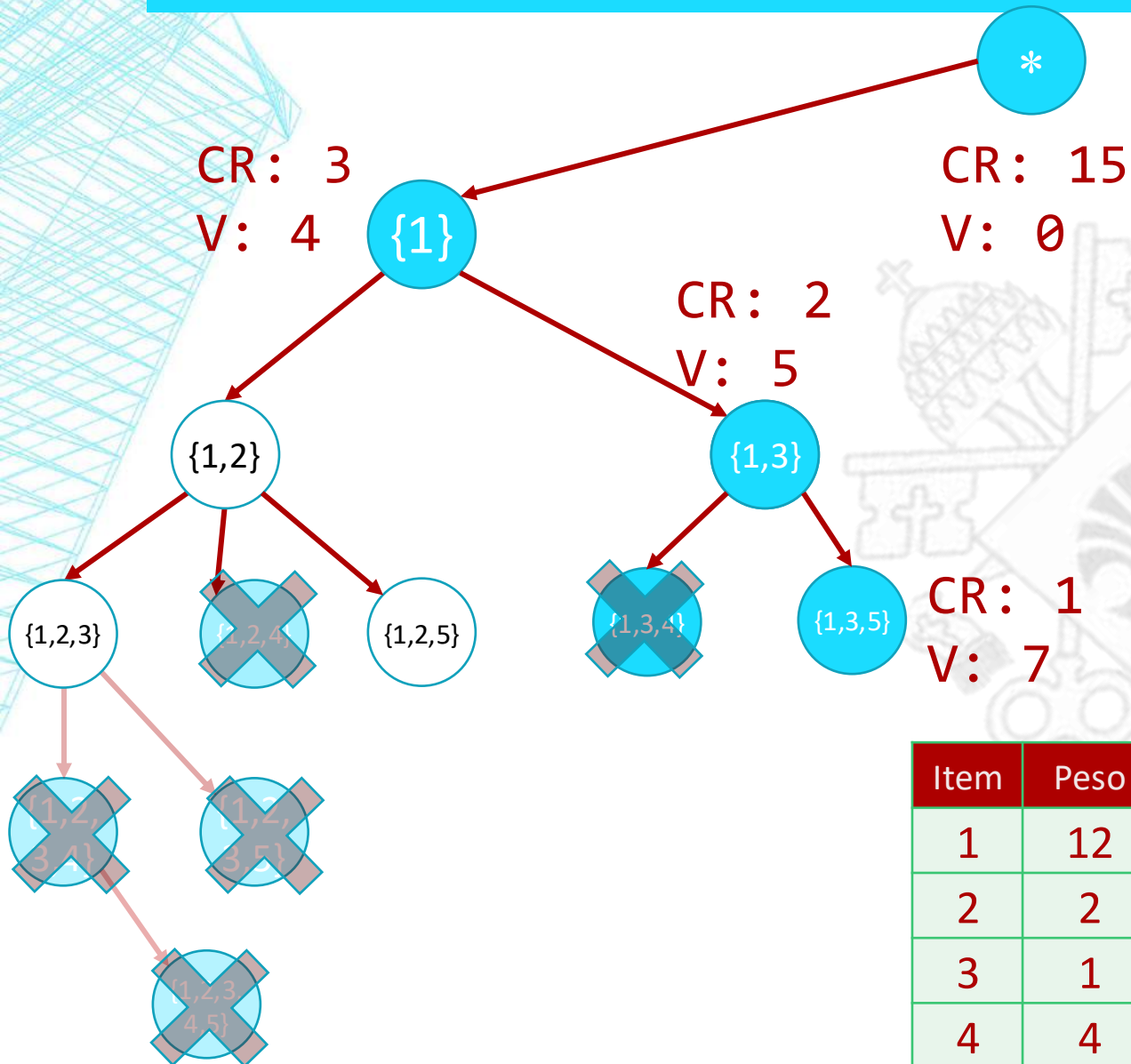
Tipos de Critérios:

2) Poda Estimada (Heurística):

- Baseia-se em estimativas, palpites inteligentes ou funções de probabilidade.
- **Trade-off:** Acelera drasticamente a busca, mas abre mão da corretude absoluta (pode acidentalmente cortar a melhor solução).

MOCHILA: PODA GARANTIDA

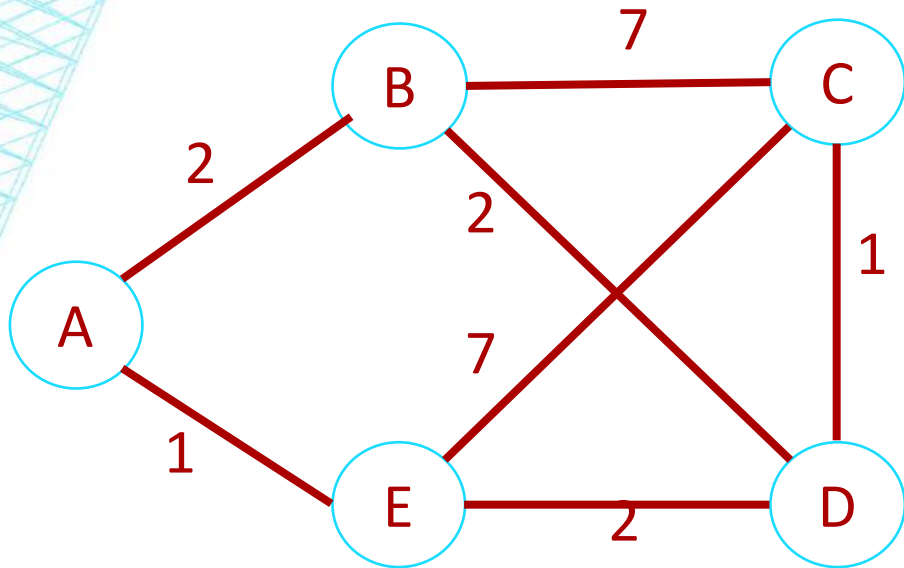
MeLhor: 8



Item	Peso	Valor
1	12	4
2	2	2
3	1	1
4	4	10
5	1	2

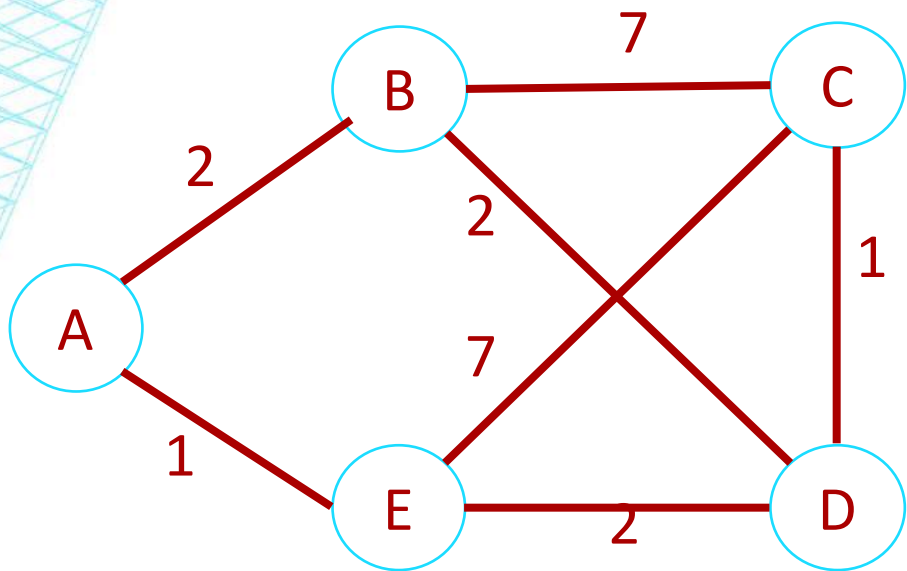
PODA ESTIMADA

- Considere o conjunto de cidades abaixo no problema do Caixeiro Viajante:



PODA ESTIMADA

- Soma de todos os caminhos: 22
- Média por caminho: 3,14



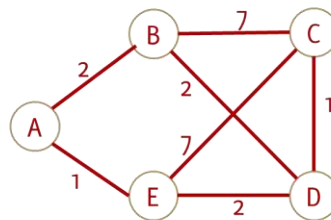
CAIXEIRO: ÁRVORE DE COMBINAÇÕES



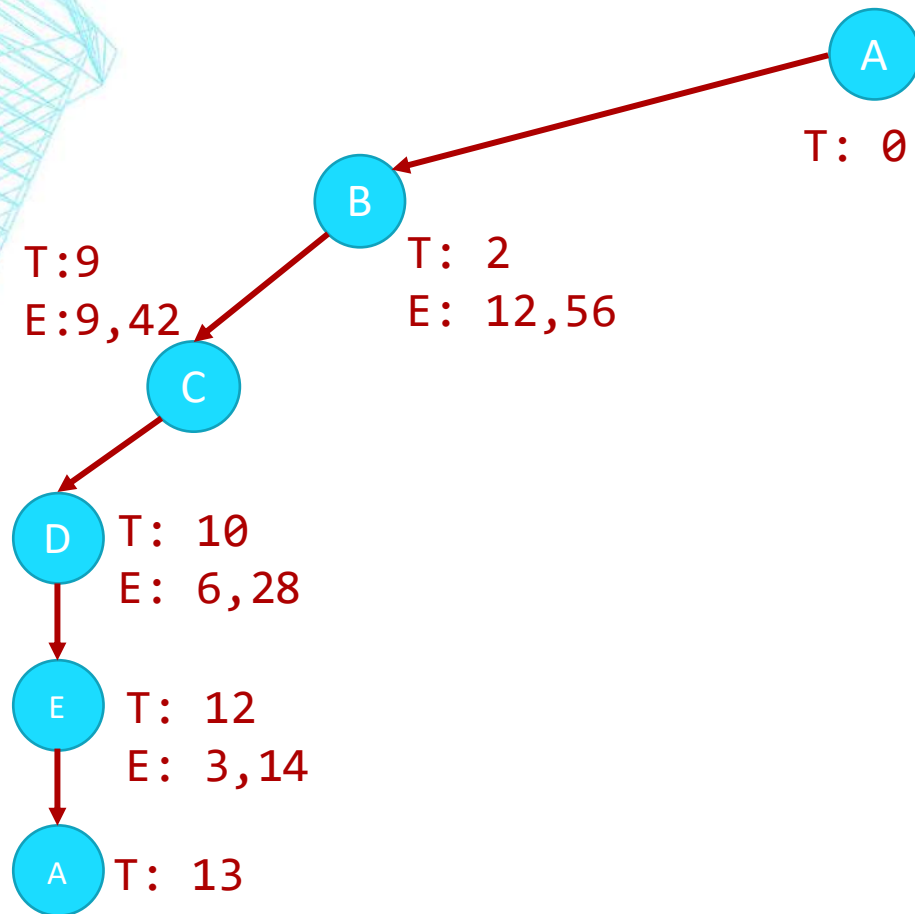
T: 0

Caminho saindo de A e terminando em A

- 5 vértices = 5 arestas
- Média por caminho = 3,14
- Esperado (E) = $5 \times 3,14 = 15,7$
- A cada aresta escolhida subtraímos 3,14 desse montante
- Total (T) = custo total (ou real) do caminho

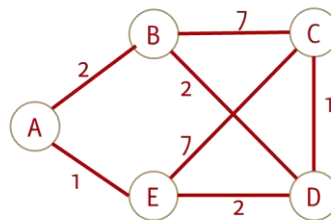


CAIXEIRO: ÁRVORE DE COMBINAÇÕES

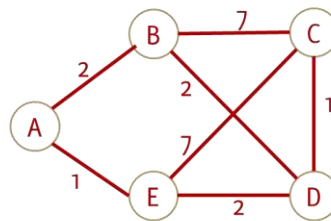
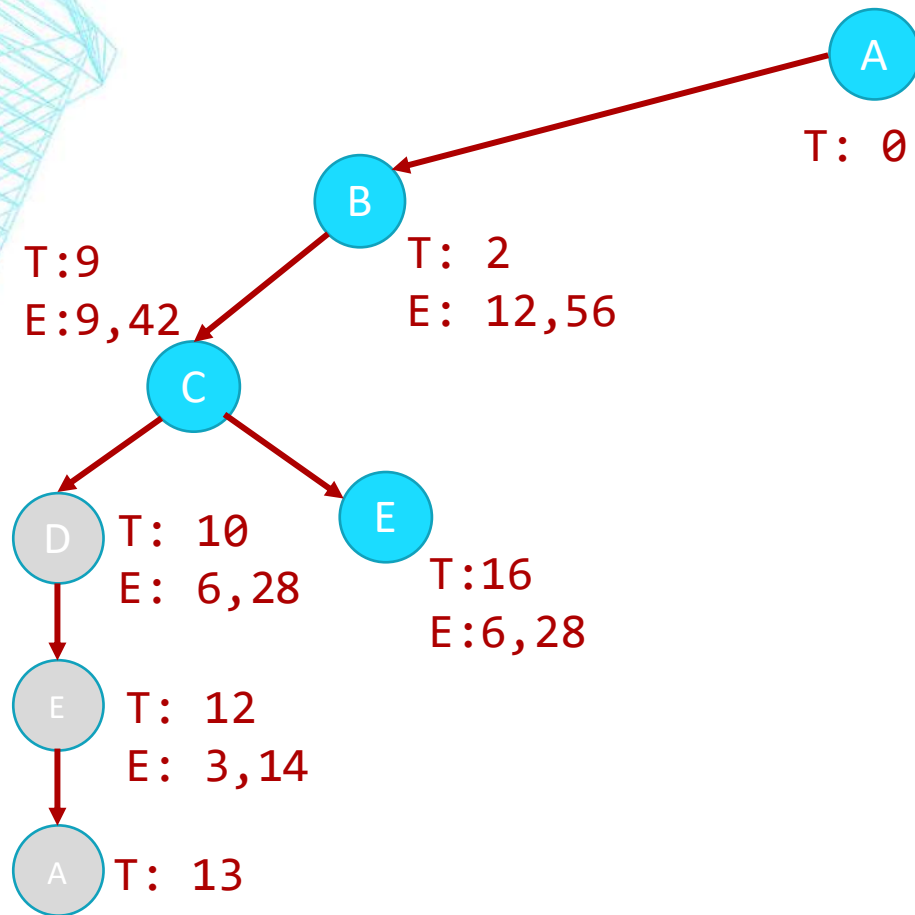


Caminho saindo de A e terminando em A

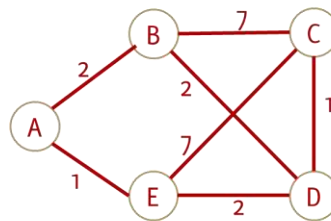
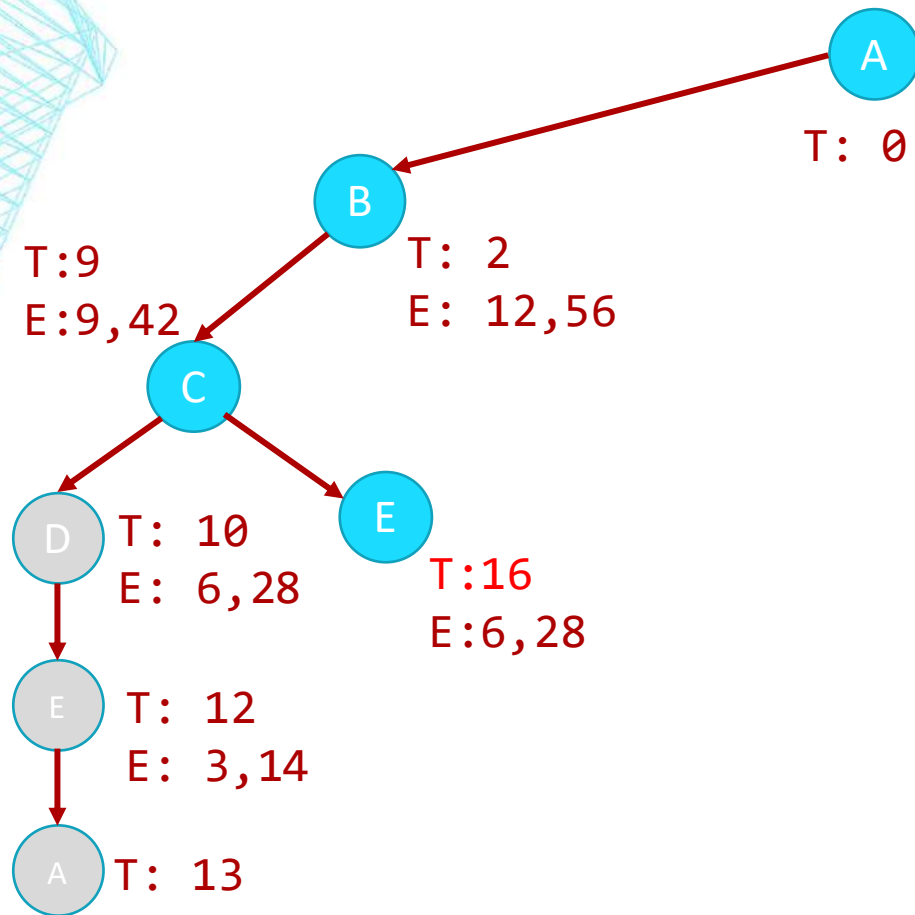
- 5 vértices = 5 arestas
- Média por caminho = 3,14
- Esperado (E) = $5 \times 3,14 = 15,7$
- A cada aresta escolhida subtraímos 3,14 desse montante
- Total (T) = custo total (ou real) do caminho



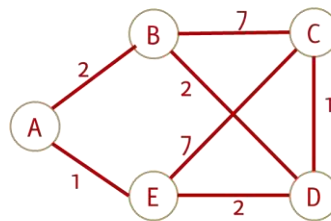
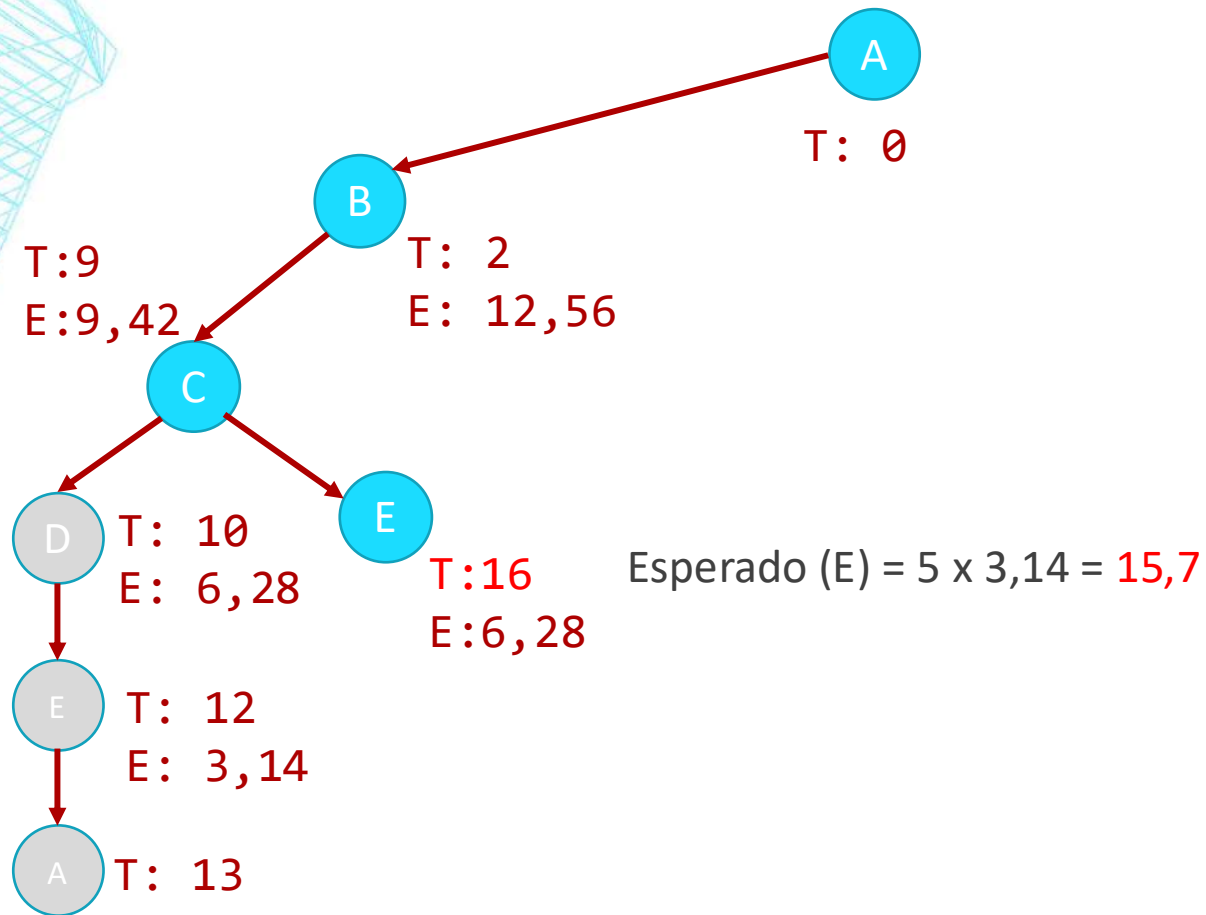
CAIXEIRO: ÁRVORE DE COMBINAÇÕES



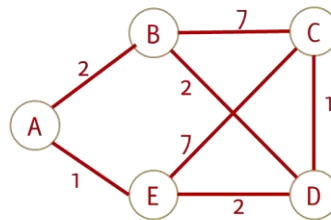
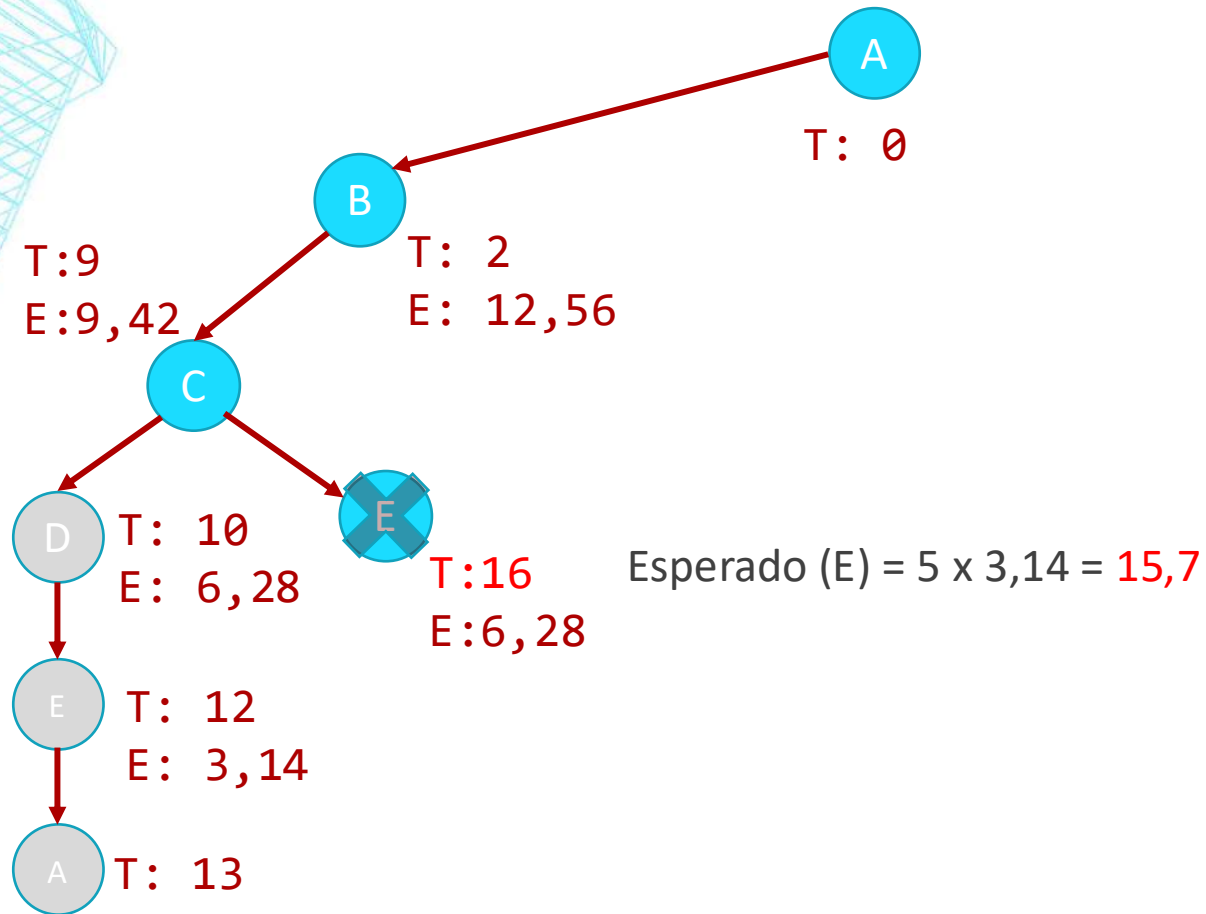
CAIXEIRO: ÁRVORE DE COMBINAÇÕES



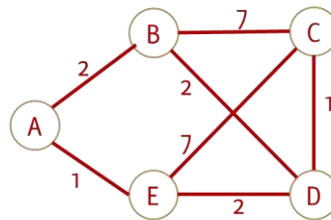
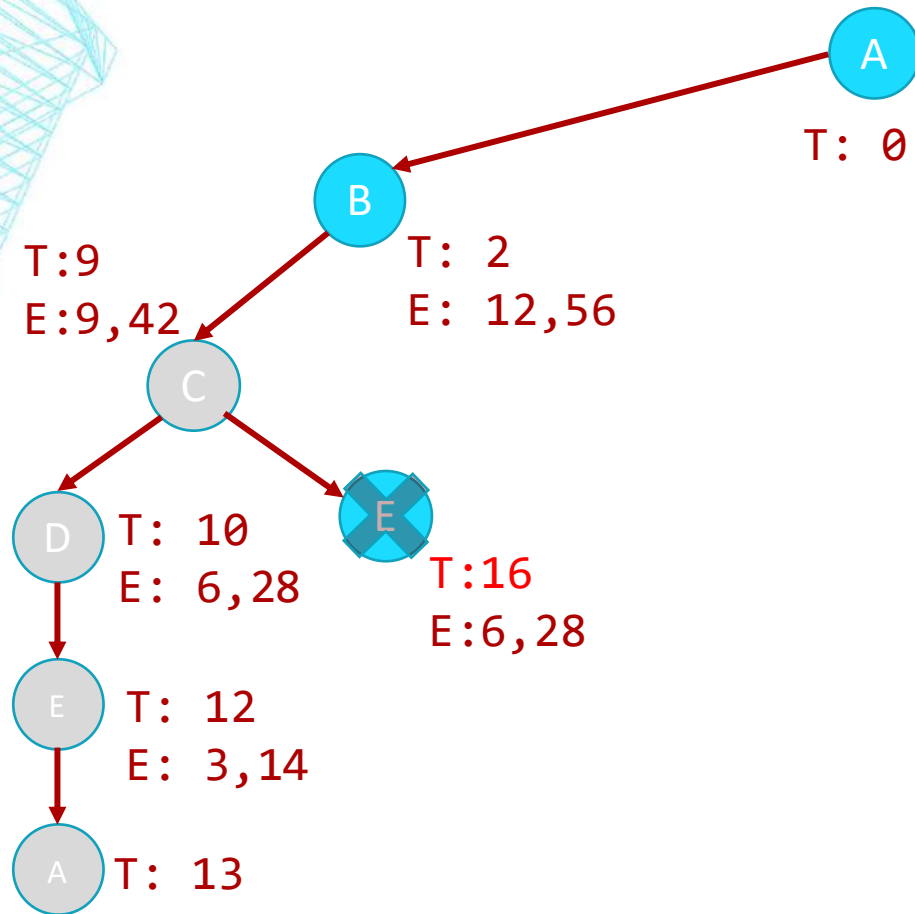
CAIXEIRO: ÁRVORE DE COMBINAÇÕES



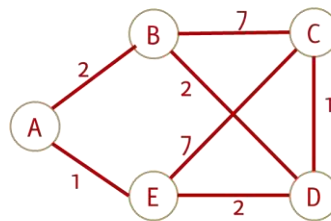
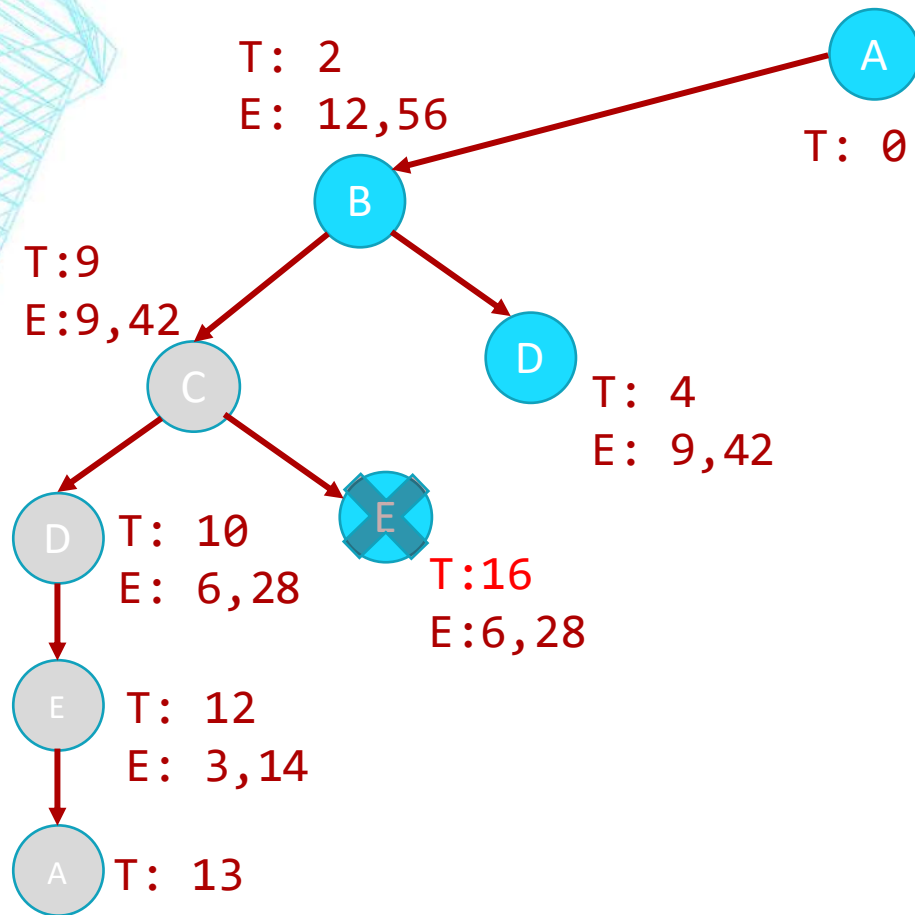
CAIXEIRO: ÁRVORE DE COMBINAÇÕES



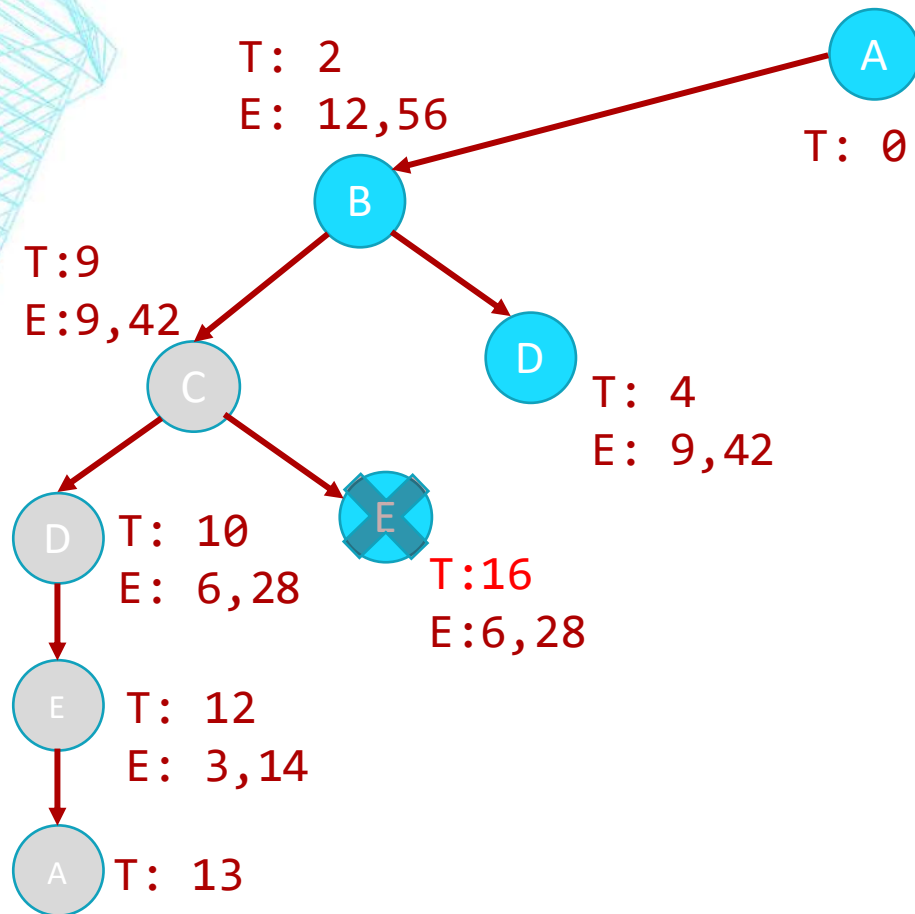
CAIXEIRO: ÁRVORE DE COMBINAÇÕES



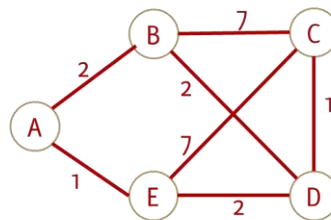
CAIXEIRO: ÁRVORE DE COMBINAÇÕES



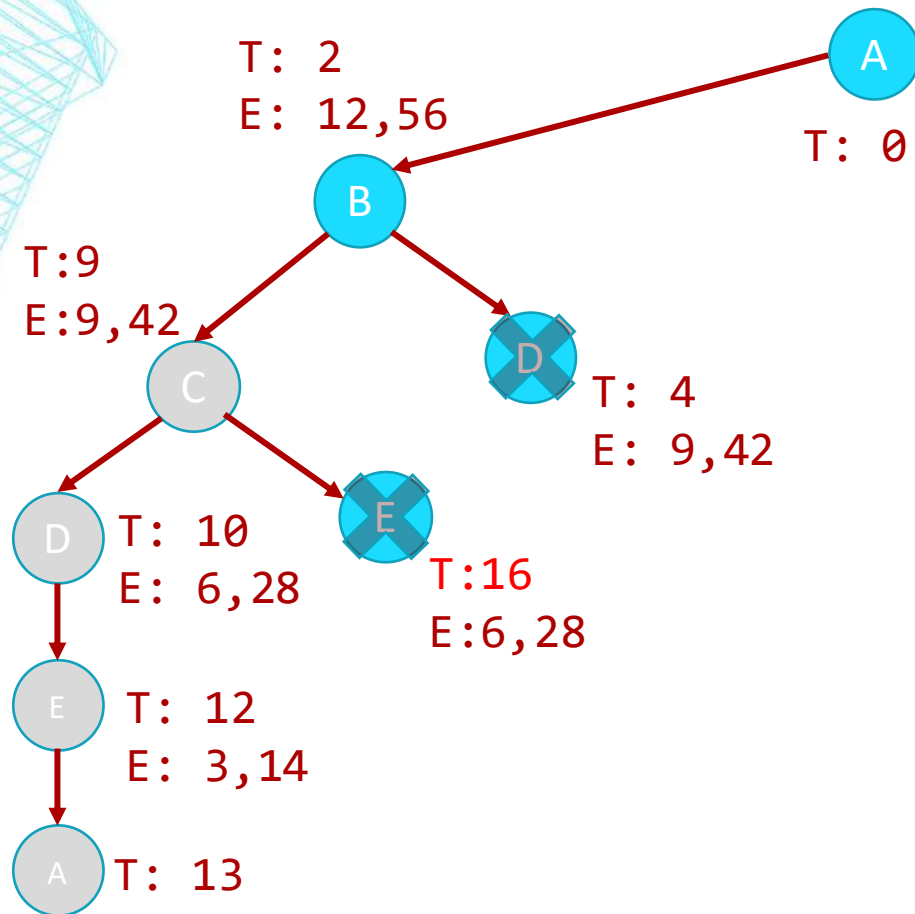
CAIXEIRO: ÁRVORE DE COMBINAÇÕES



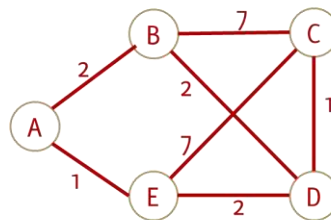
- *Best so far* = 13
- $A \rightarrow B \rightarrow D = 4$
 - Faltam mais 3 arestas:
 - $3 \times 3,14 = 9,42$
 - $9,42 + 4 = 13,42$ (pior que 13)
- Logo não seguiremos nesse caminho.



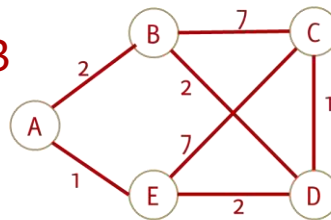
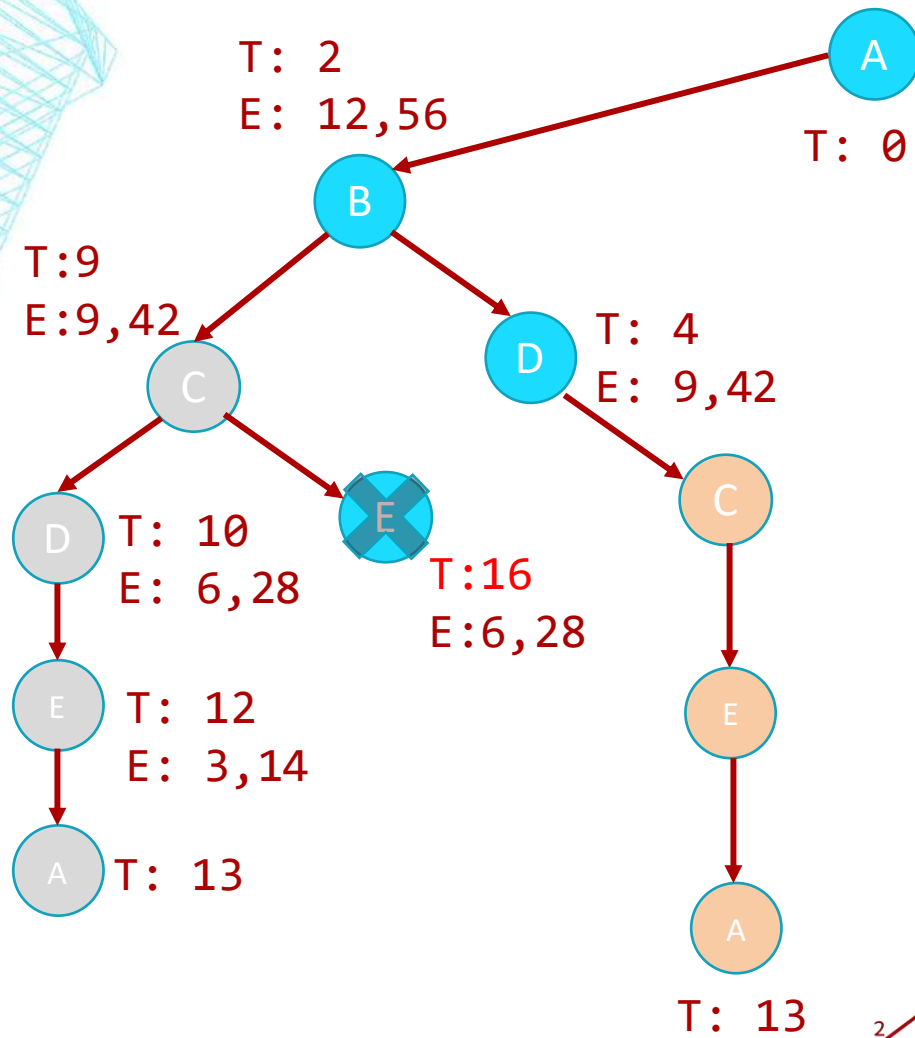
CAIXEIRO: ÁRVORE DE COMBINAÇÕES



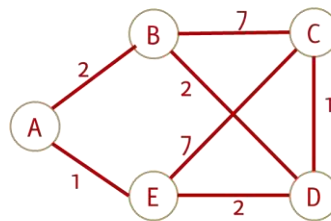
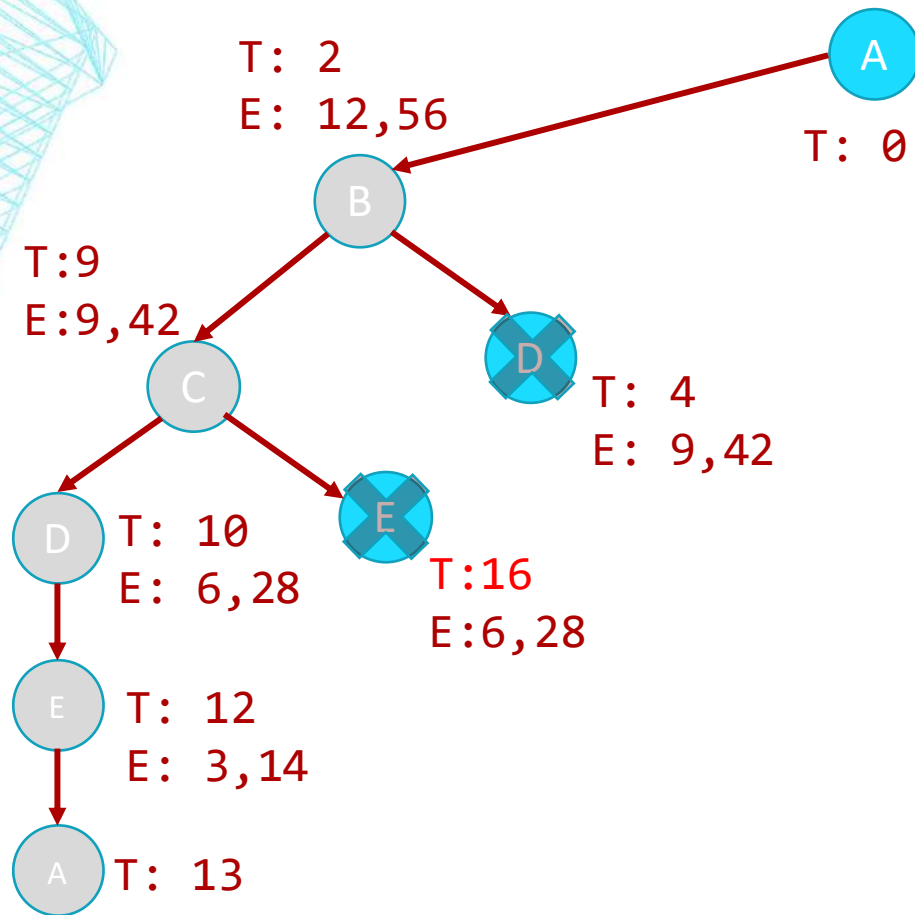
- *Best so far* = 13
- $A \rightarrow B \rightarrow D = 4$
 - Faltam mais 3 arestas:
 - $3 \times 3,14 = 9,42$
 - $9,42 + 4 = 13,42$ (pior que 13)
- Logo não seguiremos nesse caminho.



CAIXEIRO: ÁRVORE DE COMBINAÇÕES

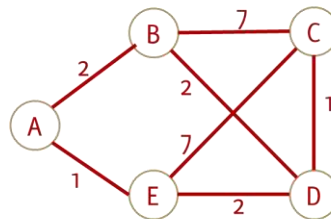
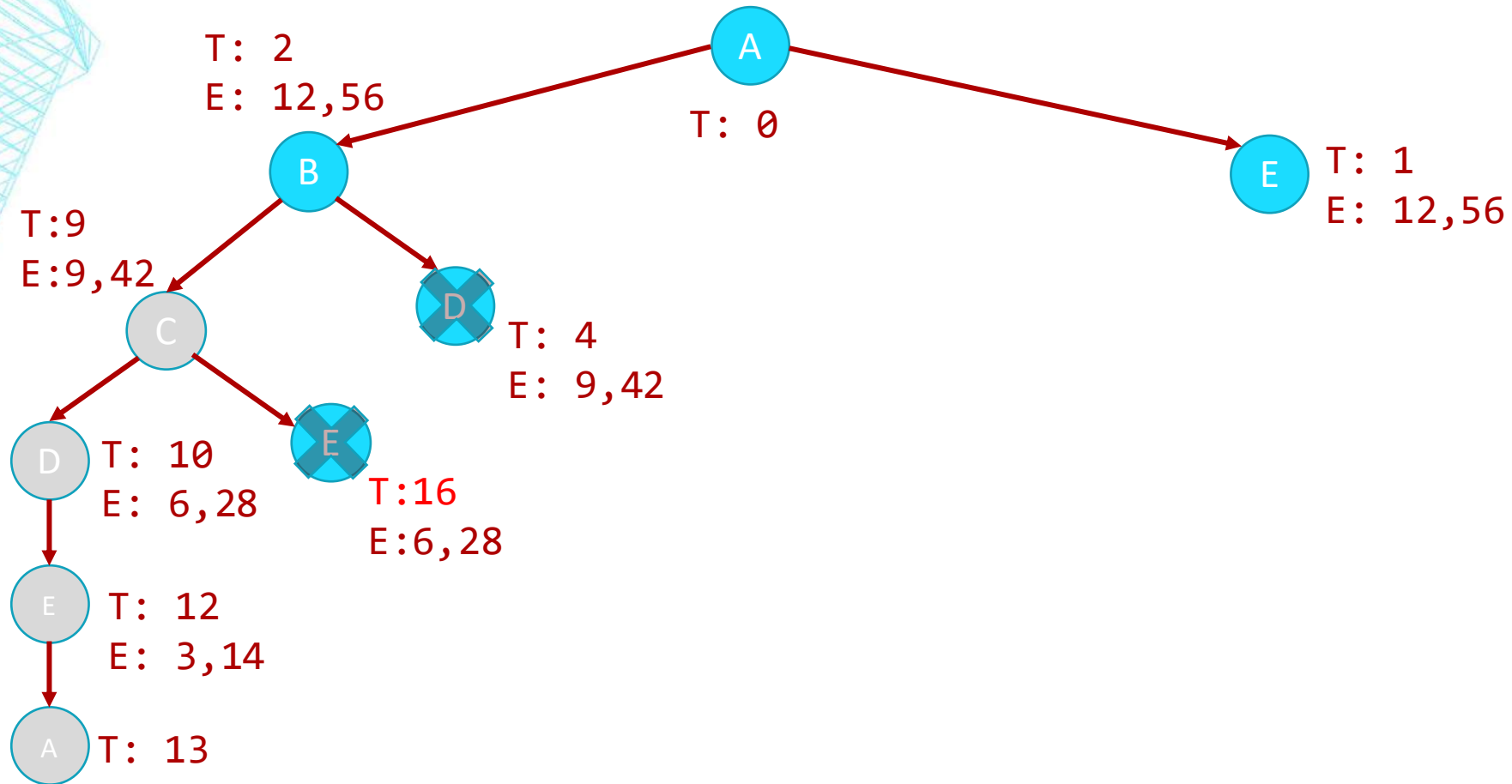


CAIXEIRO: ÁRVORE DE COMBINAÇÕES



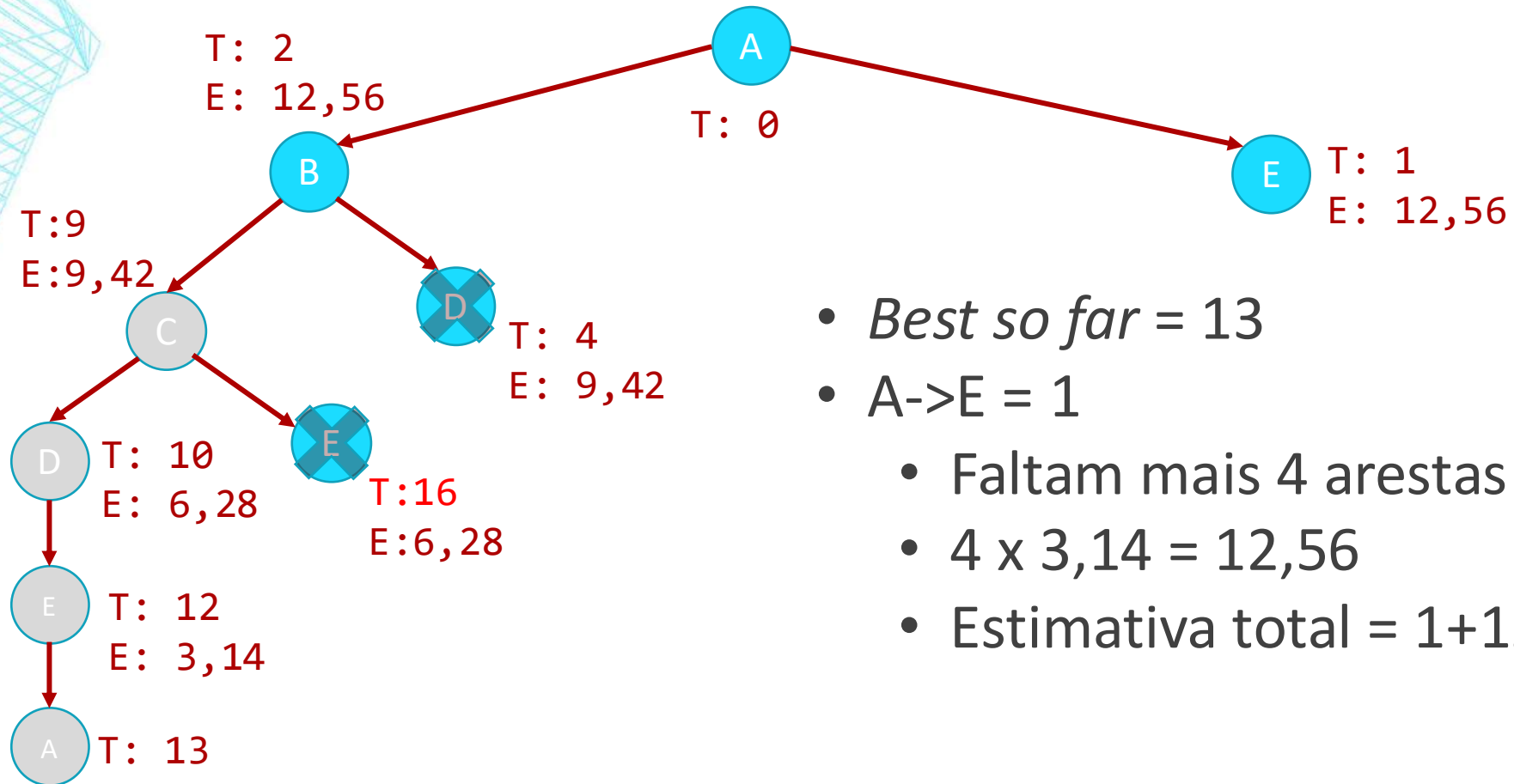
CAIXEIRO: ÁRVORE DE COMBINAÇÕES

Margem: 1

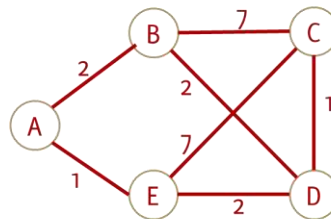


CAIXEIRO: ÁRVORE DE COMBINAÇÕES

Margem: 1

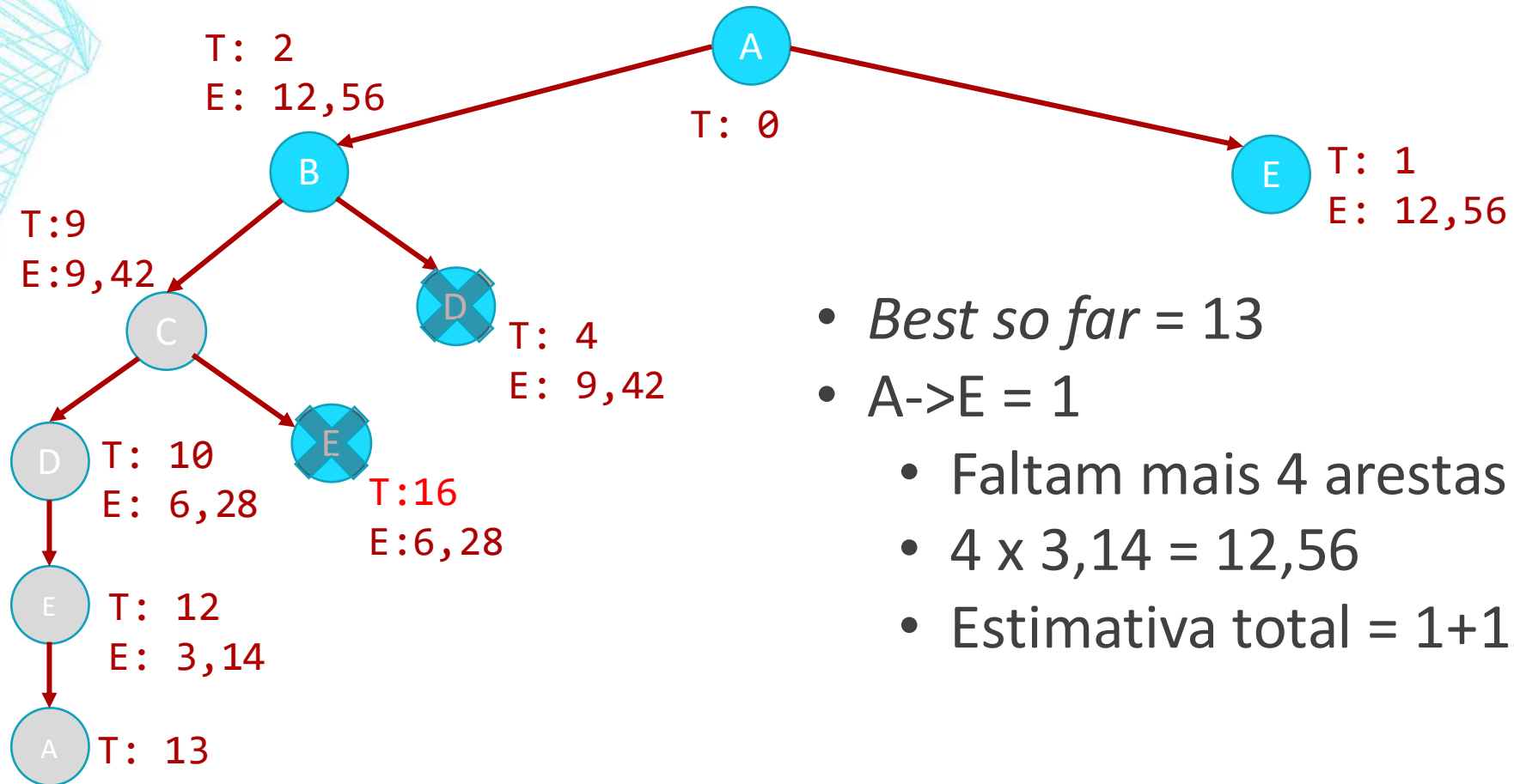


- *Best so far* = 13
- $A \rightarrow E = 1$
 - Faltam mais 4 arestas:
 - $4 \times 3,14 = 12,56$
 - Estimativa total = $1 + 12,56 = 13,56$

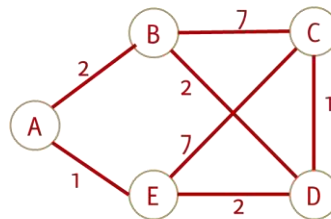


CAIXEIRO: ÁRVORE DE COMBINAÇÕES

Margem: 1



- *Best so far* = 13
- A → E = 1
 - Faltam mais 4 arestas:
 - $4 \times 3,14 = 12,56$
 - Estimativa total = $1 + 12,56 = 13,56$



O MECANISMO DE PODA (PRUNING)

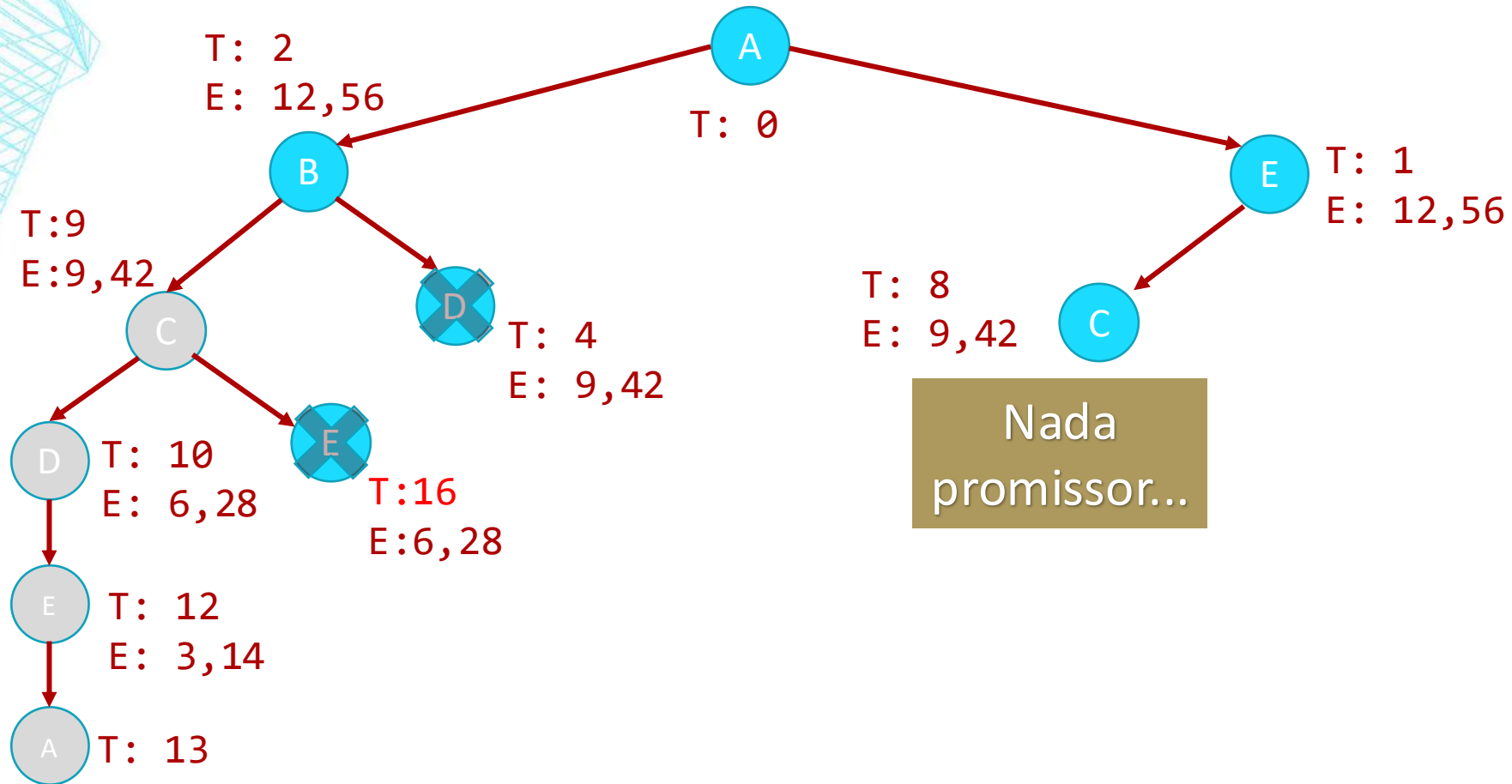
- A estimativa para esse caminho (**13,56**) é maior do que a nossa melhor solução (**13**). Se o algoritmo fosse 100% rígido, **ele teria podado o vértice E imediatamente**, logo no primeiro passo!

A Solução (A Margem):

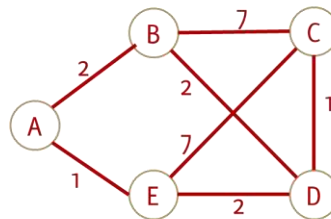
- Como o valor 3,14 é apenas uma média (uma heurística), ele não é perfeito. Pode ser que, por pura sorte, as próximas arestas a partir de E sejam incrivelmente baratas. Para não ser tão agressivo e evitar cortar uma solução ótima por causa de alguns centésimos (0,56 de diferença), o autor do algoritmo adicionou uma **Margem de Tolerância (Margem = 1)**.
- Isso significa que o novo limite de corte passa a ser: $13 + 1 = 14$.

CAIXEIRO: ÁRVORE DE COMBINAÇÕES

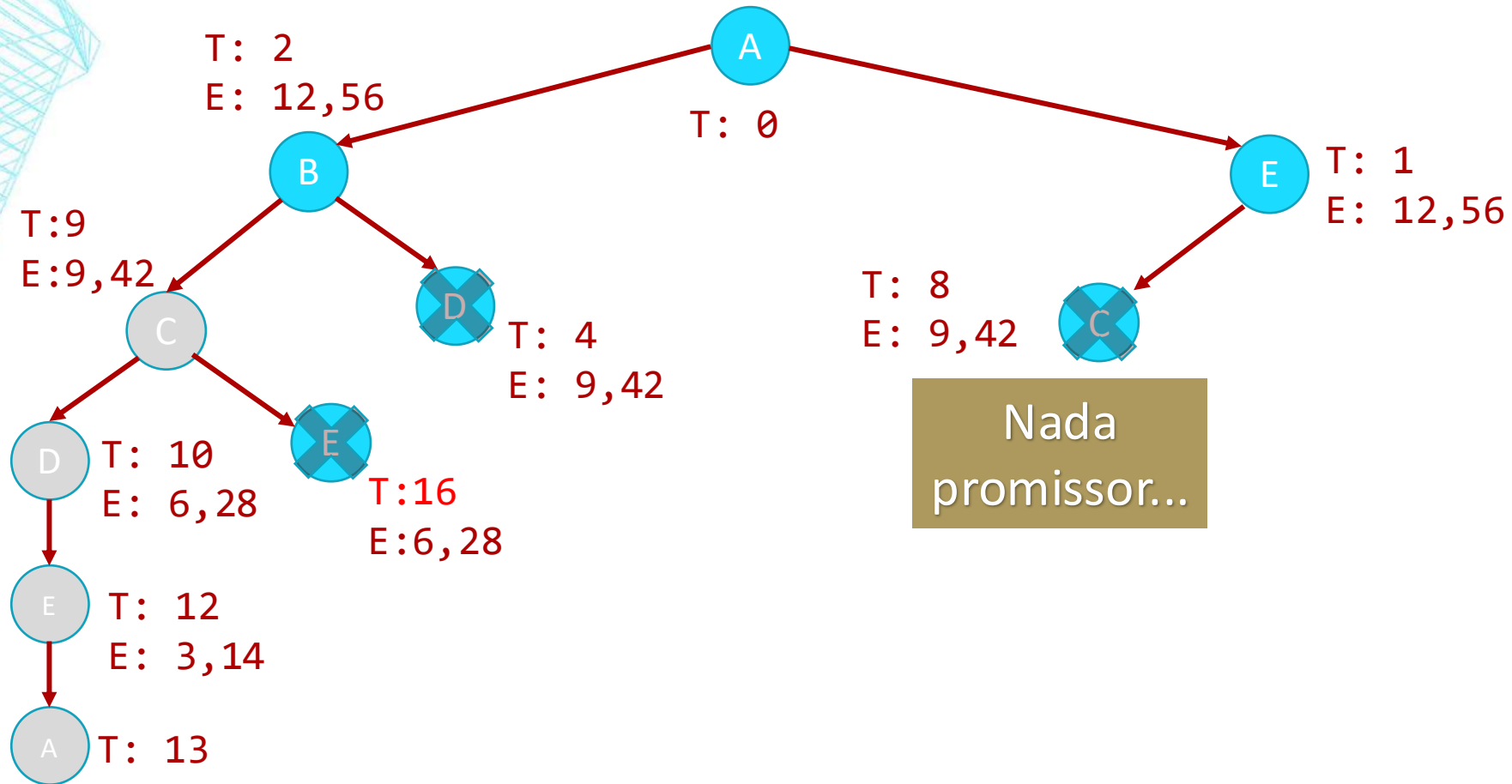
Margem: 1



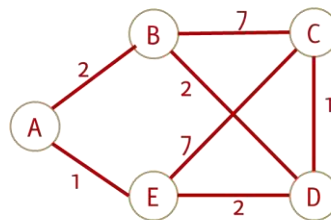
Nada promissor...



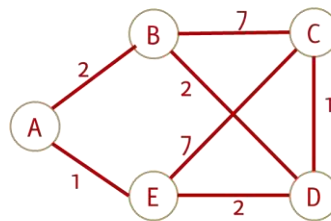
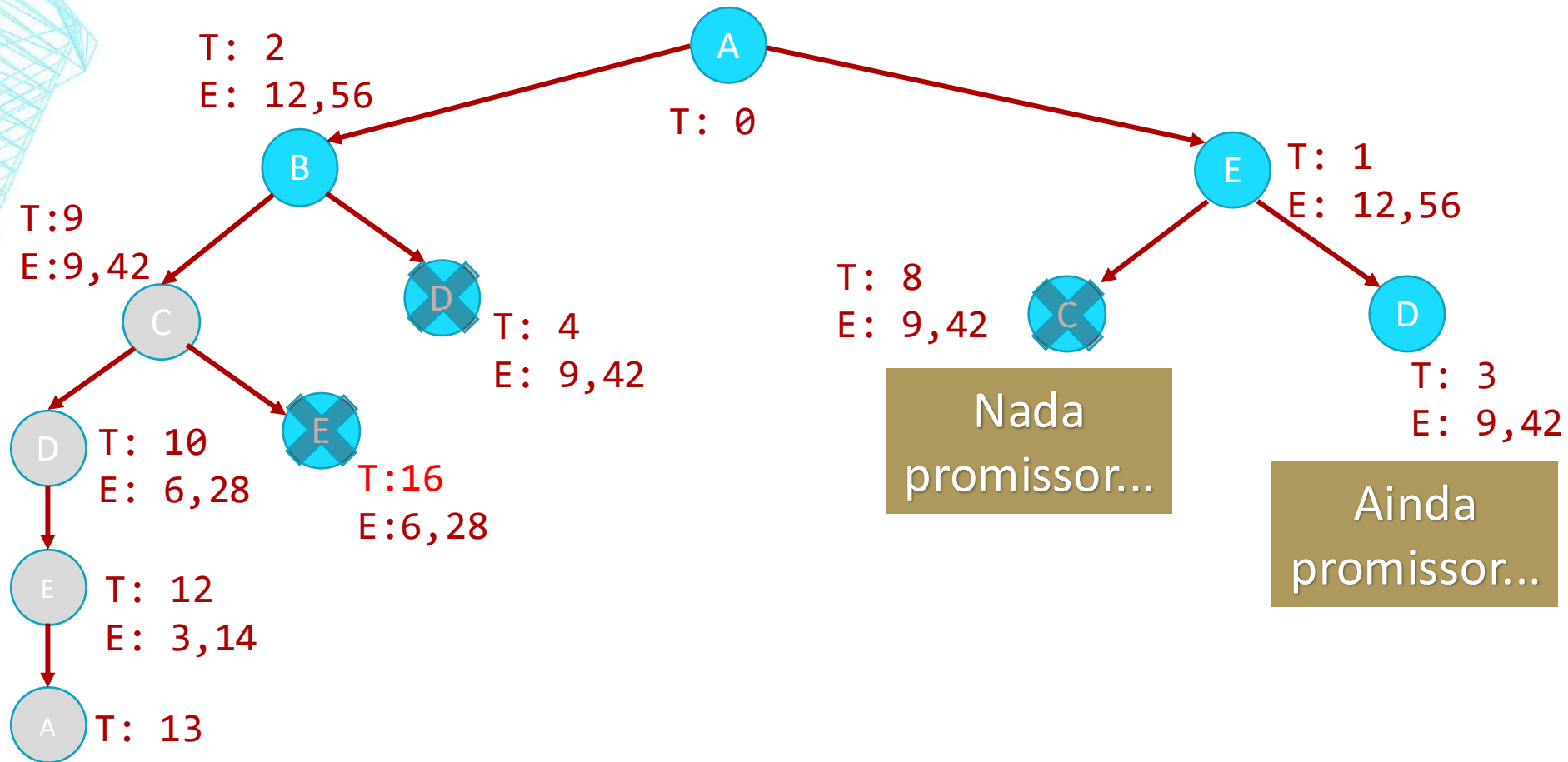
CAIXEIRO: ÁRVORE DE COMBINAÇÕES



Nada promissor...



CAIXEIRO: ÁRVORE DE COMBINAÇÕES



```

// Parâmetros Globais do Algoritmo
Média_Aresta = 3.14
Melhor_Solução = 13 // Best So Far
Margem = 1
FUNÇÃO BuscaCaixeiro(vértice_atual, T, arestas_restantes)
    // 1. Cálculo da Projeção de Custo (A Heurística)
    E = arestas_restantes * Média_Aresta
    Projeção = T + E
    // 2. O CRITÉRIO DE PODA (Branch and Bound) S
    SE Projeção > (Melhor_Solução + Margem)
        ENTÃO RETORNE FALSO // Fim da linha! O backtracking ocorre aqui.
    FIM SE
    // 3. Condição de Parada (Encontrou um ciclo completo válido)
    SE arestas_restantes == 0 ENTÃO
        SE T < Melhor_Solução ENTÃO
            Melhor_Solução = T // Atualiza o novo limite
        FIM SE
        RETORNE VERDADEIRO
    FIM SE
    // 4. Geração de Candidatos (Ramificação)
    PARA CADA próximo_vértice a partir de vértice_atual FAÇA
        SE próximo_vértice não foi visitado ENTÃO
            Marca próximo_vértice como visitado
            Novo_T = T + peso(vértice_atual, próximo_vértice)
            // Avança para o próximo nível da árvore
            BuscaCaixeiro(próximo_vértice, Novo_T, arestas_restantes - 1)
            // Desfaz o movimento para testar o próximo irmão (Retrocesso)
            Desmarca próximo_vértice
        FIM SE
    FIM PARA
FIM FUNÇÃO

```

BACKTRACKING - CONSIDERAÇÕES

- Geralmente aplicada a problemas combinatórios, para os quais não existam algoritmos eficientes.
- Recursividade provê forma simples de resolver problemas complexos.
 - **Pendência**: geração das soluções candidatas.
- Verificação de extensão deve ser eficiente.

BACKTRACKING - CONSIDERAÇÕES

- O sucesso varia bastante de problema a problema e entre instâncias do mesmo problema.
- Quanto mais soluções candidatas, mais próximo o algoritmo fica da busca exaustiva.

BACKTRACKING - CONSIDERAÇÕES

- Possível excesso de memória em problemas grandes.
- Limitações e heurísticas.
 - Poda: descarta ramos inviáveis, reduzindo bastante o tempo total.

CONSIDERAÇÕES FINAIS

